



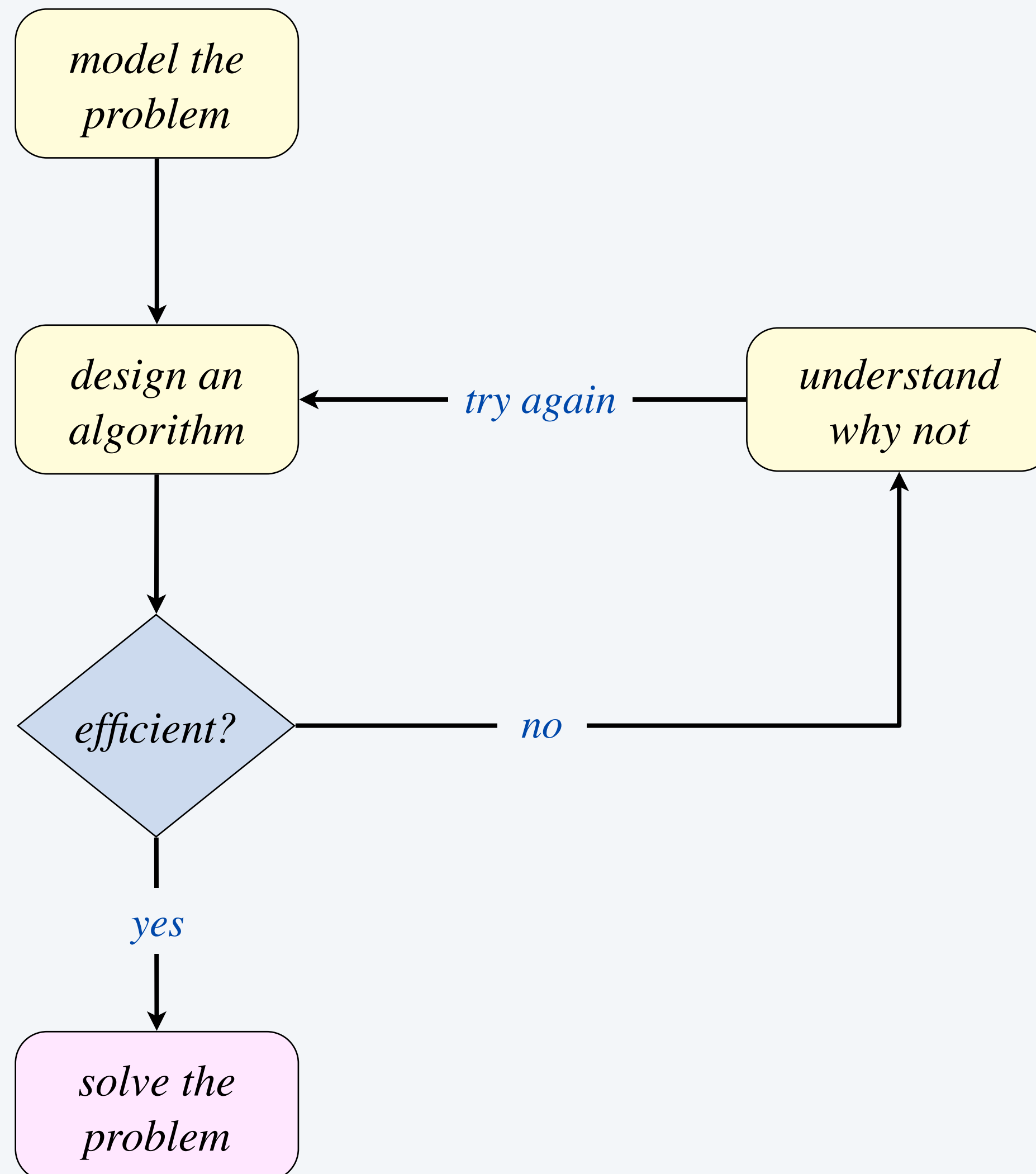
<https://algs4.cs.princeton.edu>

1.5 UNION-FIND

- *union-find data type*
- *quick-find*
- *quick-union*
- *weighted quick-union*

Subtext of today's lecture (and this course)

Steps to develop a usable algorithm to solve a computational problem.





<https://algs4.cs.princeton.edu>

1.5 UNION-FIND

- *union-find data type*
- *quick-find*
- *quick-union*
- *weighted quick-union*
- *percolation*

Union-find data type

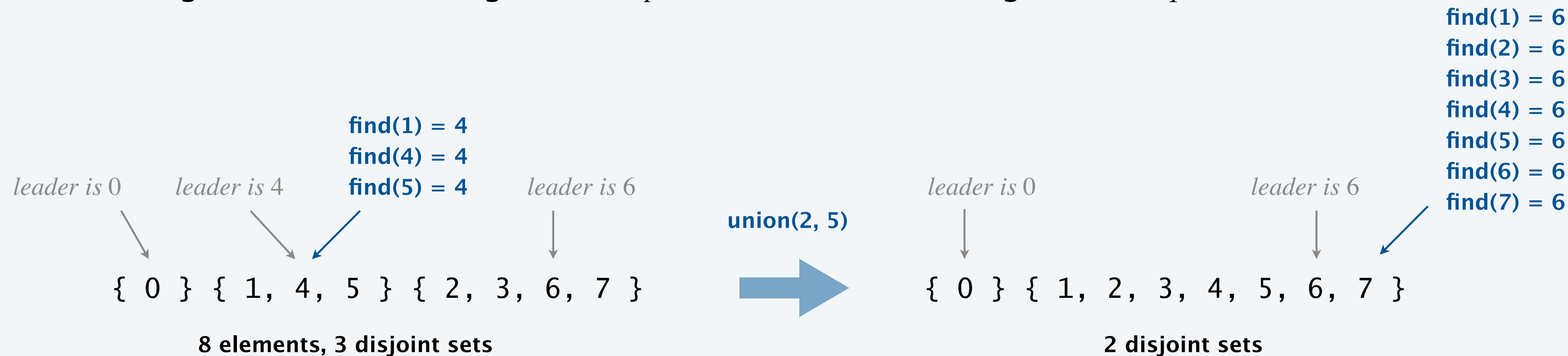
Disjoint sets. A collection of sets containing n elements, with each element in exactly one set.

Leader. Each set designates one of its elements as **leader** to uniquely identify the set.

*no restriction on which element is designated leader
(but leader of a set can't change unless the set changes)*

Find. Return the leader of the set containing element p . *typical use case: are two elements in the same set ?*

Union. Merge the set containing element p with the set containing element q .



Union–find data type: API

Goal. Design an **efficient** union–find data type.

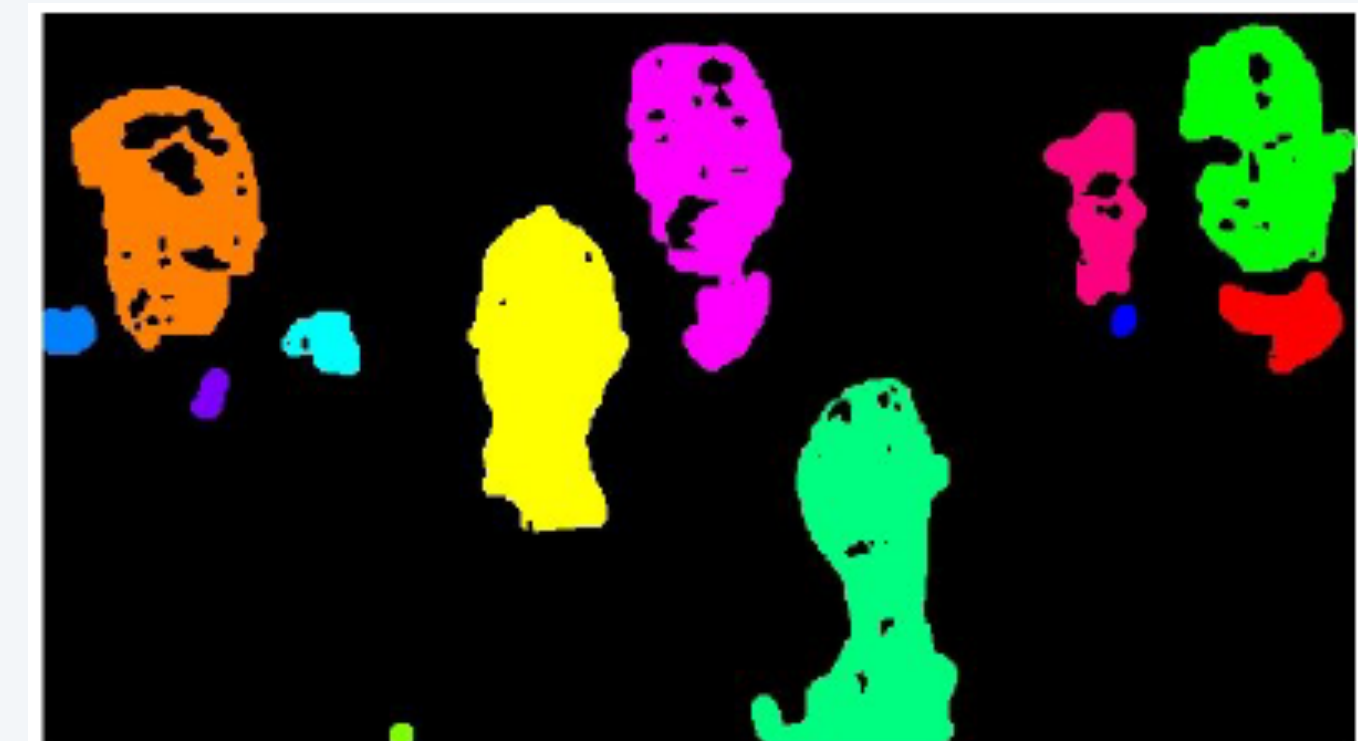
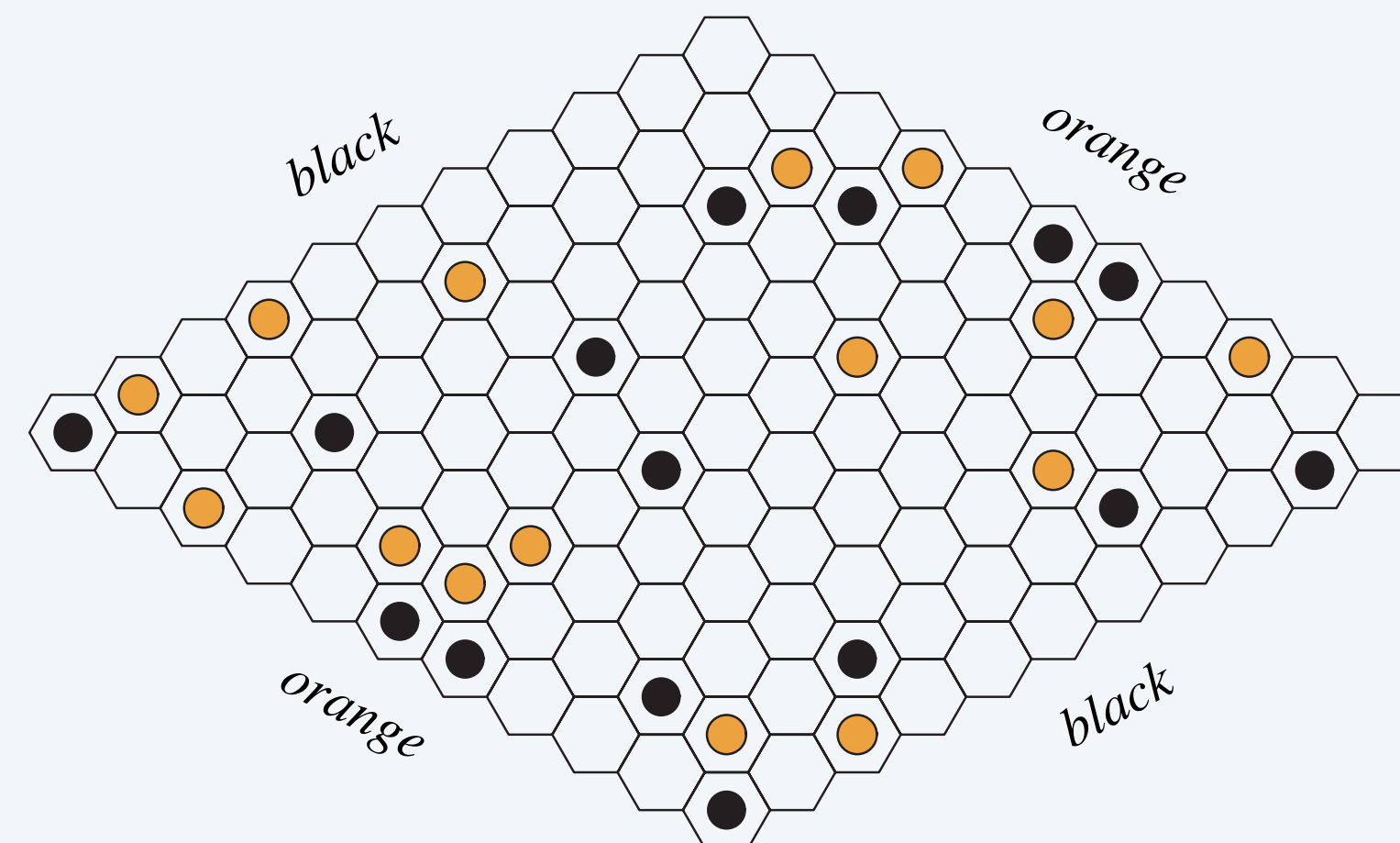
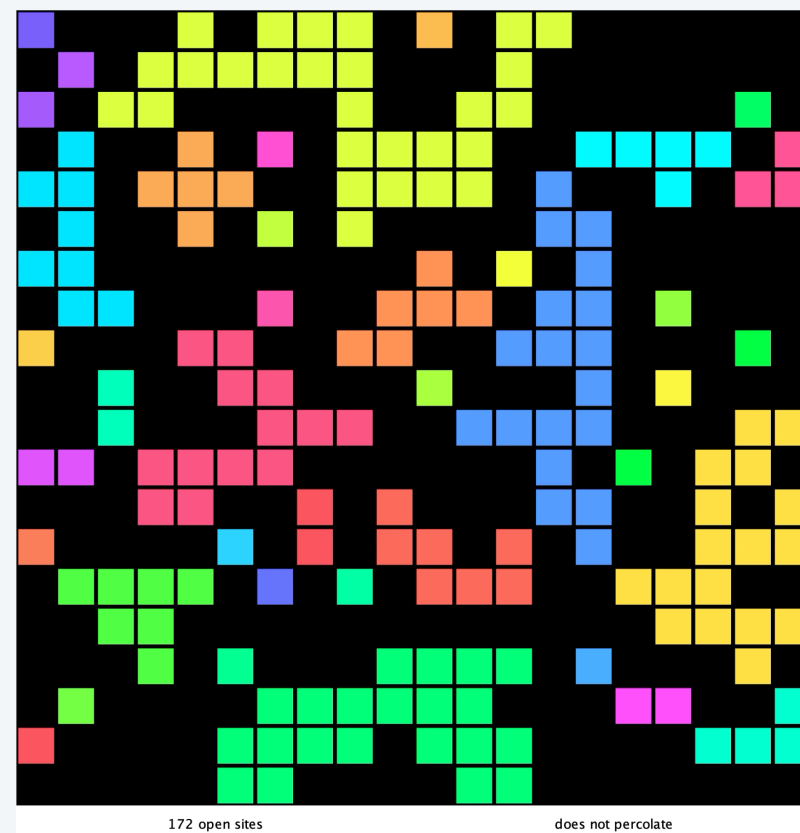
- Simplifying assumption: the n elements are named $0, 1, \dots, n - 1$.
- The `union()` and `find()` operations can be intermixed.
- Number of elements n can be huge.
- Number of operations m can be huge.

<code>public class UF</code>	description
<code>UF(int n)</code>	<i>initialize with n singleton sets (0 to $n - 1$)</i>
<code>void union(int p, int q)</code>	<i>merge sets containing elements p and q</i>
<code>int find(int p)</code>	<i>return the leader of set containing element p</i>

Union–find data type: applications

Disjoint sets can represent:

- Clusters of conducting sites in a composite system. ← see Assignment 1 (Percolation)
- Connected components in a graph. ← see Kruskal's algorithm (MST lecture)
- Interlinked friends in a social network.
- Interconnected devices in a mobile network.
- Equivalent variable names in a Fortran program.
- Adjoining stones of the same color in the game of Hex.
- Contiguous pixels corresponding to same feature in a digital image.





<https://algs4.cs.princeton.edu>

1.5 UNION-FIND

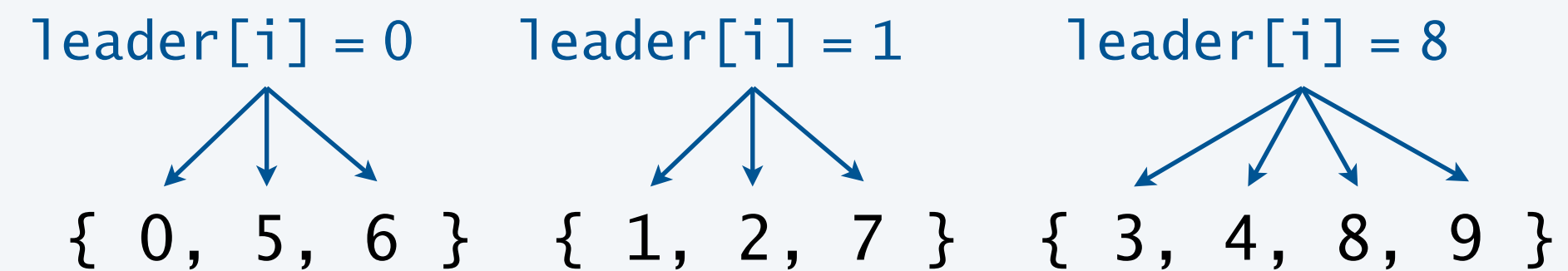
- *union-find data type*
- *quick-find*
- *quick-union*
- *weighted quick-union*

Quick-find

Data structure.

- Integer array `leader[]` of length `n`.
- Interpretation: `leader[i]` is the leader of the set containing element `i`.

	0	1	2	3	4	5	6	7	8	9
<code>leader[]</code>	0	1	1	8	8	0	0	1	8	8



10 elements, 3 disjoint sets

Q. How to implement `find(p)`?

A. Easy, just return `leader[p]`.

Quick-find

Data structure.

- Integer array `leader[]` of length `n`.
- Interpretation: `leader[i]` is the leader of the set containing element `i`.

`union(6, 1)`

	0	1	2	3	4	5	6	7	8	9
<code>leader[]</code>	1	1	1	8	8	1	1	1	8	8

*performance issue:
many array entries can change*

Q. How to implement `union(p, q)`?

A. Change all array entries whose value is `leader[p]` to `leader[q]`. ← *or vice versa*

Quick-find: Java implementation

```
public class QuickFindUF {  
    private int[] leader;
```

```
    public QuickFindUF(int n) {  
        leader = new int[n];  
        for (int i = 0; i < n; i++)  
            leader[i] = i;  
    }
```

← initialize leader of each element to itself
(n array accesses)

```
    public int find(int p) {  
        return leader[p];  
    }
```

← return the leader of p
(1 array access)

```
    public void union(int p, int q) {  
        int pLeader = leader[p];  
        int qLeader = leader[q];  
        for (int i = 0; i < leader.length; i++)  
            if (leader[i] == pLeader)  
                leader[i] = qLeader;  
    }
```

← change all array entries whose value
is $\text{leader}[p]$ to $\text{leader}[q]$
($\geq n$ array accesses)

将所有的节点的父亲都
改成新的根节点

Quick-find is too slow

Cost model. Number of array accesses (for read or write).

algorithm	initialize	union	find
quick-find	n	n	1

worst-case number of array accesses (ignoring leading coefficient)

Union is too expensive. Processing any sequence of m `union()` operations on n elements takes $\geq mn$ array accesses.


quadratic in input size!

Ex. Performing 10^9 `union()` operations on 10^9 elements might take 30 years.



<https://algs4.cs.princeton.edu>

1.5 UNION-FIND

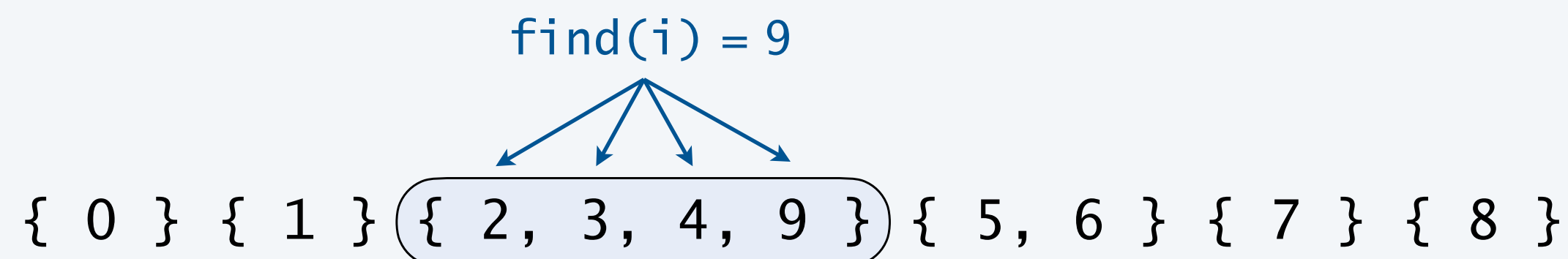
- *union-find data type*
- *quick-find*
- *quick-union*
- *weighted quick-union*

Quick-union

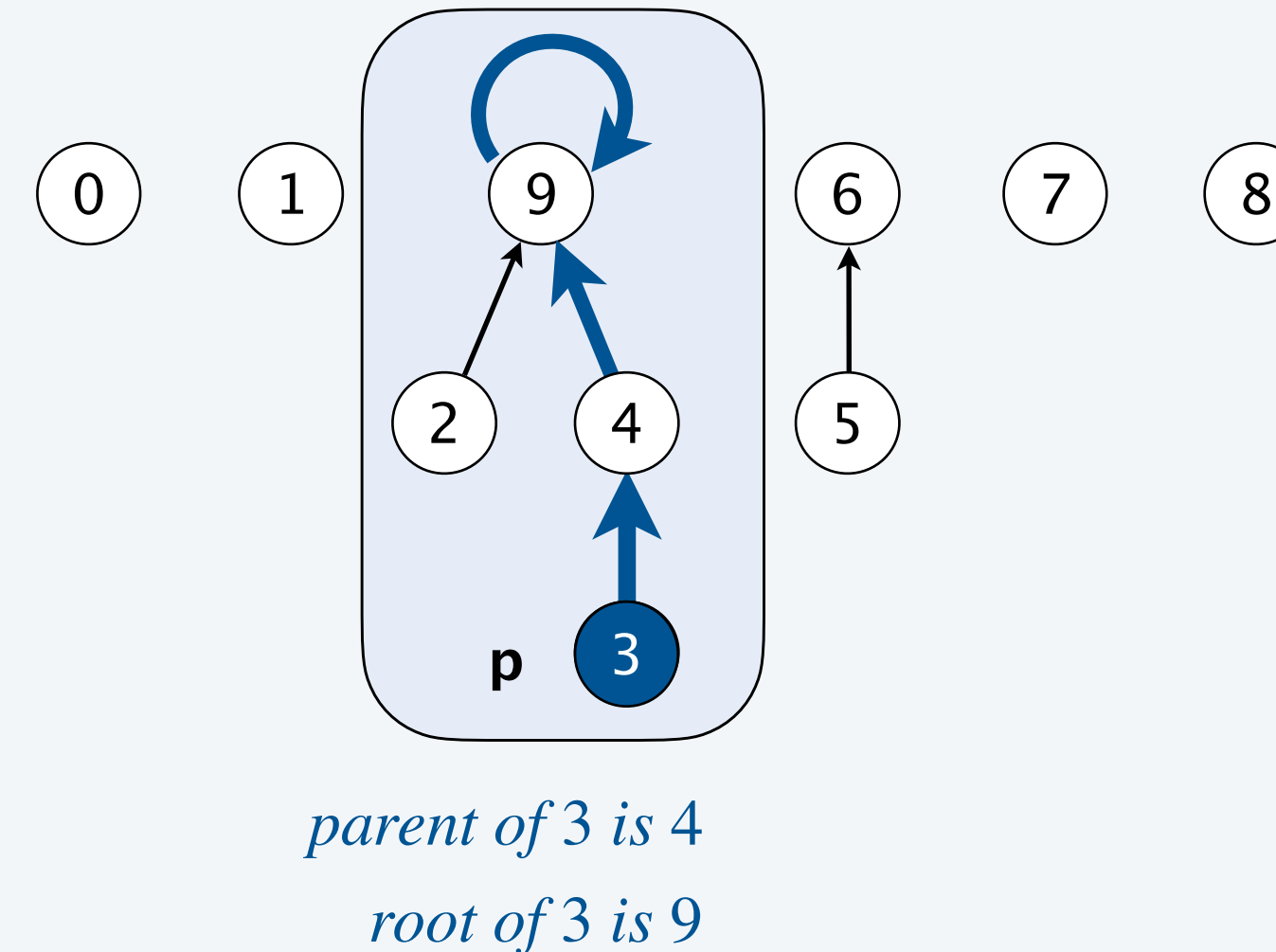
Data structure: Forest-of-trees.

- Interpretation: elements in one rooted tree correspond to one set.
- Integer array `parent[]` of length `n`, where `parent[i]` is parent of element `i` in tree.

	0	1	2	3	4	5	6	7	8	9
<code>parent[]</code>	0	1	9	4	9	6	6	7	8	9



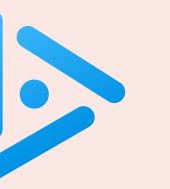
10 elements, 6 disjoint sets (6 trees)



Q. How to implement `find(p)`?

A. Use tree roots as leaders $\Rightarrow$ return **root** of tree containing `p`.

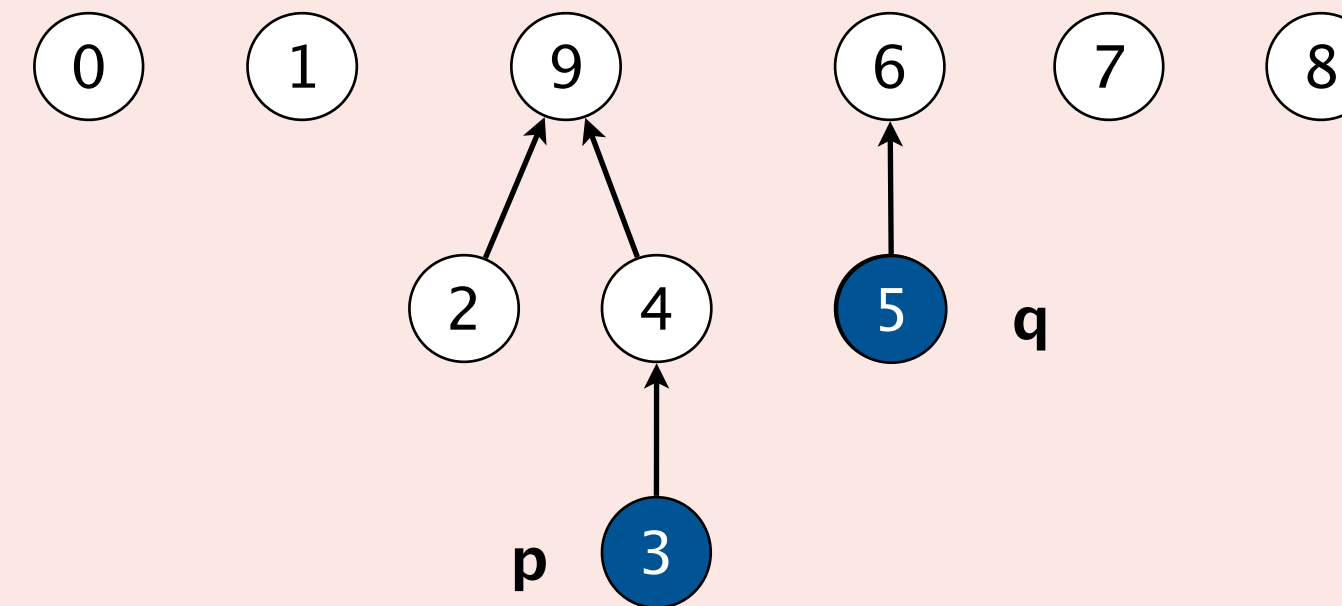
Union-find: quiz 1



Data structure: Forest-of-trees.

- Interpretation: elements in one rooted tree correspond to one set.
- Integer array `parent[]` of length `n`, where `parent[i]` is parent of element `i` in tree.

	0	1	2	3	4	5	6	7	8	9
parent[]	0	1	9	4	9	6	6	7	8	9



Which is **not** a valid way to implement `union(3, 5)` ?

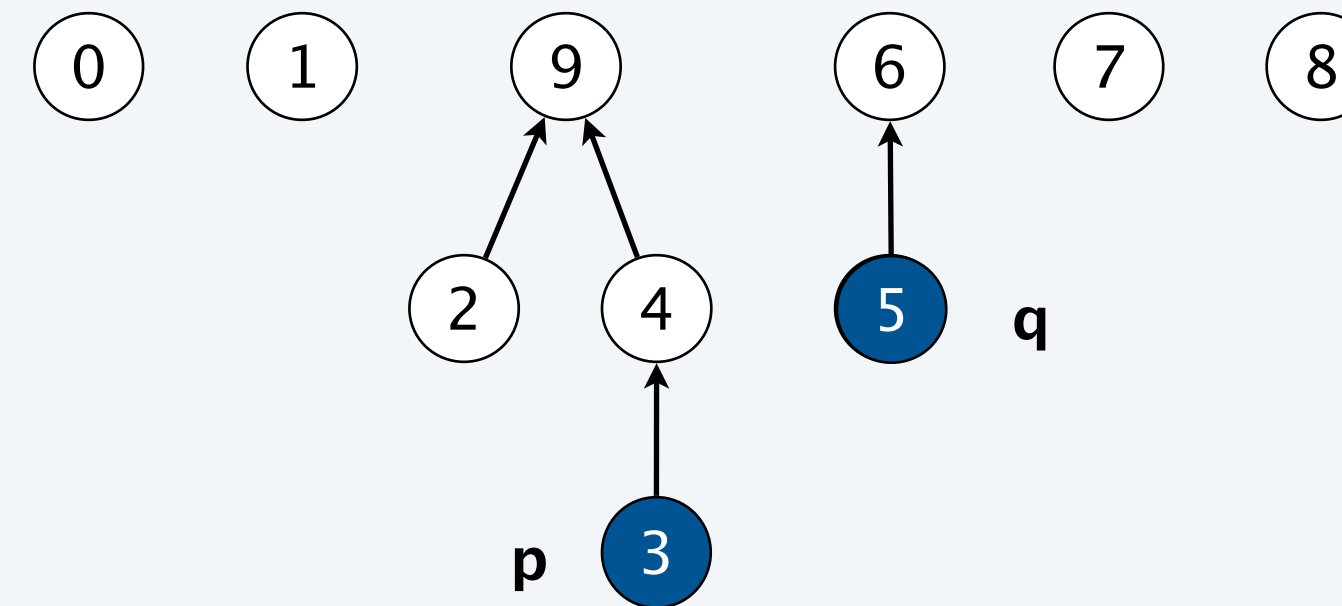
- A. Set `parent[6] = 9`.
- B. Set `parent[9] = 6`.
- C. Set `parent[3] = 5`.
- D. Set `parent[2] = parent[3] = parent[4] = parent[9] = 6`.

Quick-union

Data structure: Forest-of-trees.

- Interpretation: elements in one rooted tree correspond to one set.
- Integer array `parent[]` of length `n`, where `parent[i]` is parent of element `i` in tree.

	0	1	2	3	4	5	6	7	8	9
<code>union(3, 5)</code>	0	1	9	4	9	6	6	7	8	9



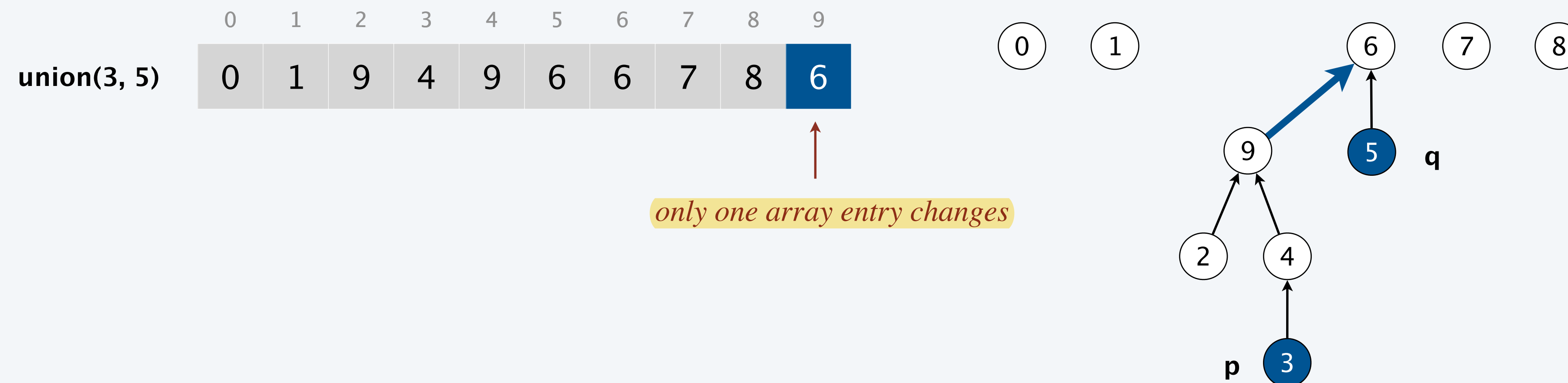
Q. How to implement `union(p, q)`?

A. Set `parent[p's root] = q's root`. $\longleftarrow$ *or vice versa*

Quick-union

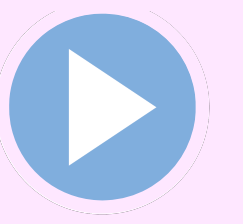
Data structure: Forest-of-trees.

- Interpretation: elements in one rooted tree correspond to one set.
- Integer array `parent[]` of length `n`, where `parent[i]` is parent of element `i` in tree.



Q. How to implement `union(p, q)`?

A. Set `parent[p's root] = q's root.` ← *or vice versa*



Quick-union: Java implementation

```
public class QuickUnionUF {  
    private int[] parent;
```

```
    public QuickUnionUF(int n) {  
        parent = new int[n];  
        for (int i = 0; i < n; i++)  
            parent[i] = i;  
    }
```

*set parent of each element to itself
(to create forest of n singleton trees)*

```
    public int find(int p) {  
        while (p != parent[p])  
            p = parent[p];  
        return p;  
    }
```

*follow parent pointers until reach root;
return resulting root*

默认是p挂到q

```
    public void union(int p, int q) {  
        int root1 = find(p);  
        int root2 = find(q);  
        parent[root1] = root2;  
    }
```

quick union只修改根节点的父亲

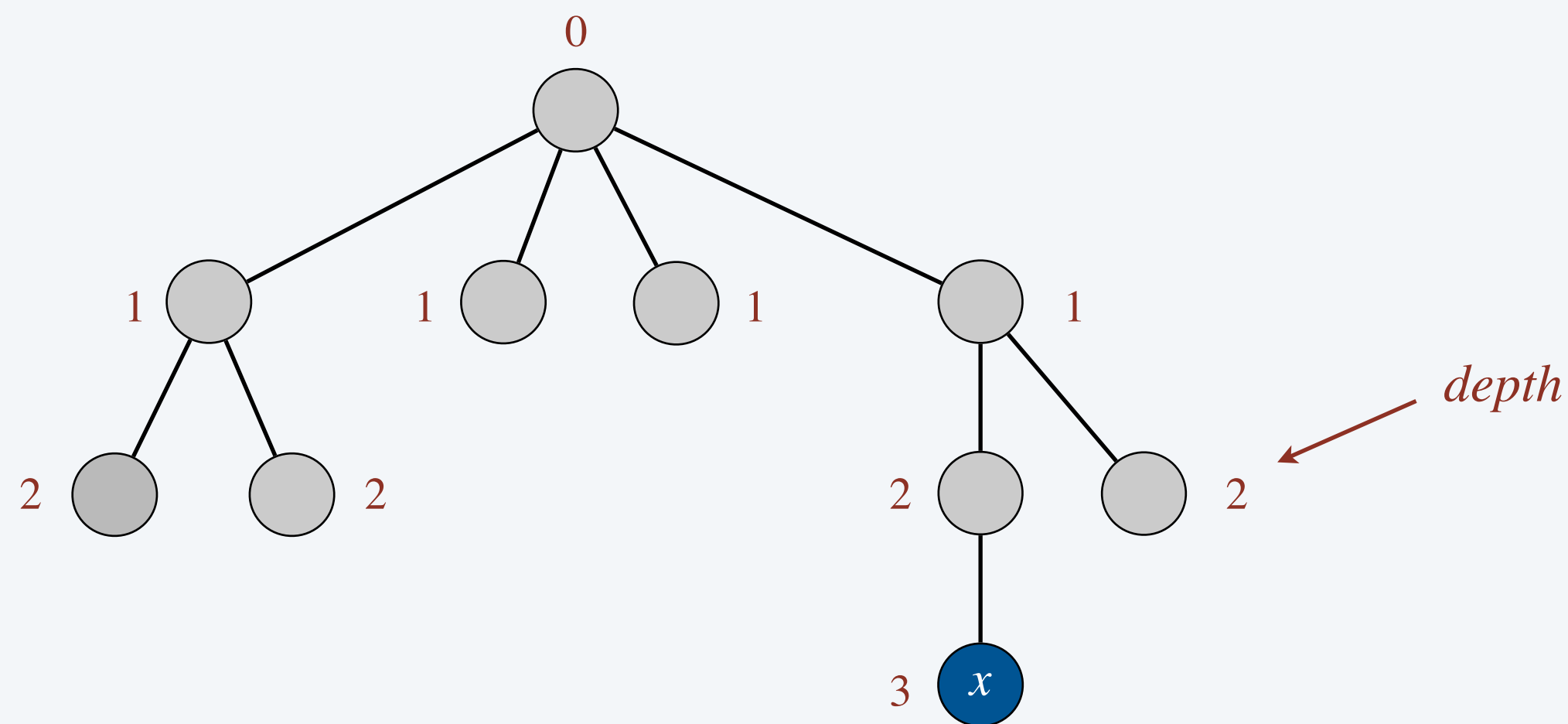
link root of p to root of q

Quick-union analysis

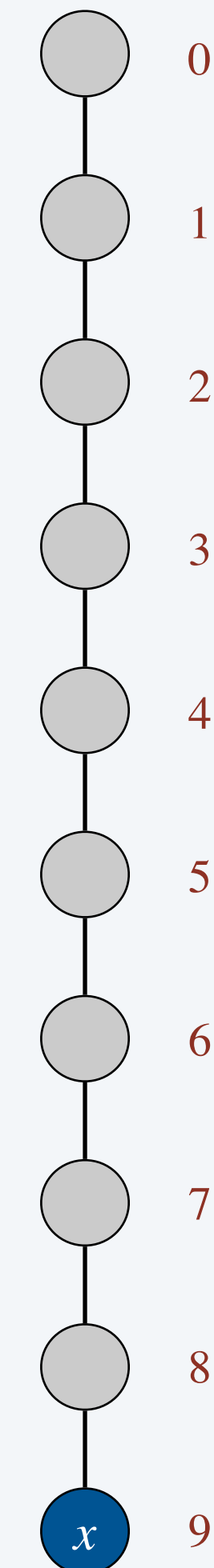
Cost model. Number of array accesses (for read or write).

Running time.

- `union()` takes constant time, given two roots.
- `find()` takes time proportional to **depth** of node in tree.



$\text{depth}(x) = 3$



worst-case depth = $n-1$

Quick-union analysis

Cost model. Number of array accesses (for read or write).

Running time.

- `union()` takes constant time, given two roots.
- `find()` takes time proportional to **depth** of node in tree.

algorithm	initialize	union	find
quick-find	n	n	1
quick-union	n	n	n

worst-case number of array accesses (ignoring leading coefficient)

Union and find are too expensive (if trees get tall). Processing some sequences of m `union()` and `find()` operations on n elements takes $\geq mn$ array accesses.

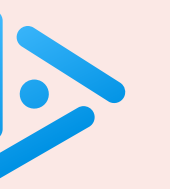

quadratic in input size !



<https://algs4.cs.princeton.edu>

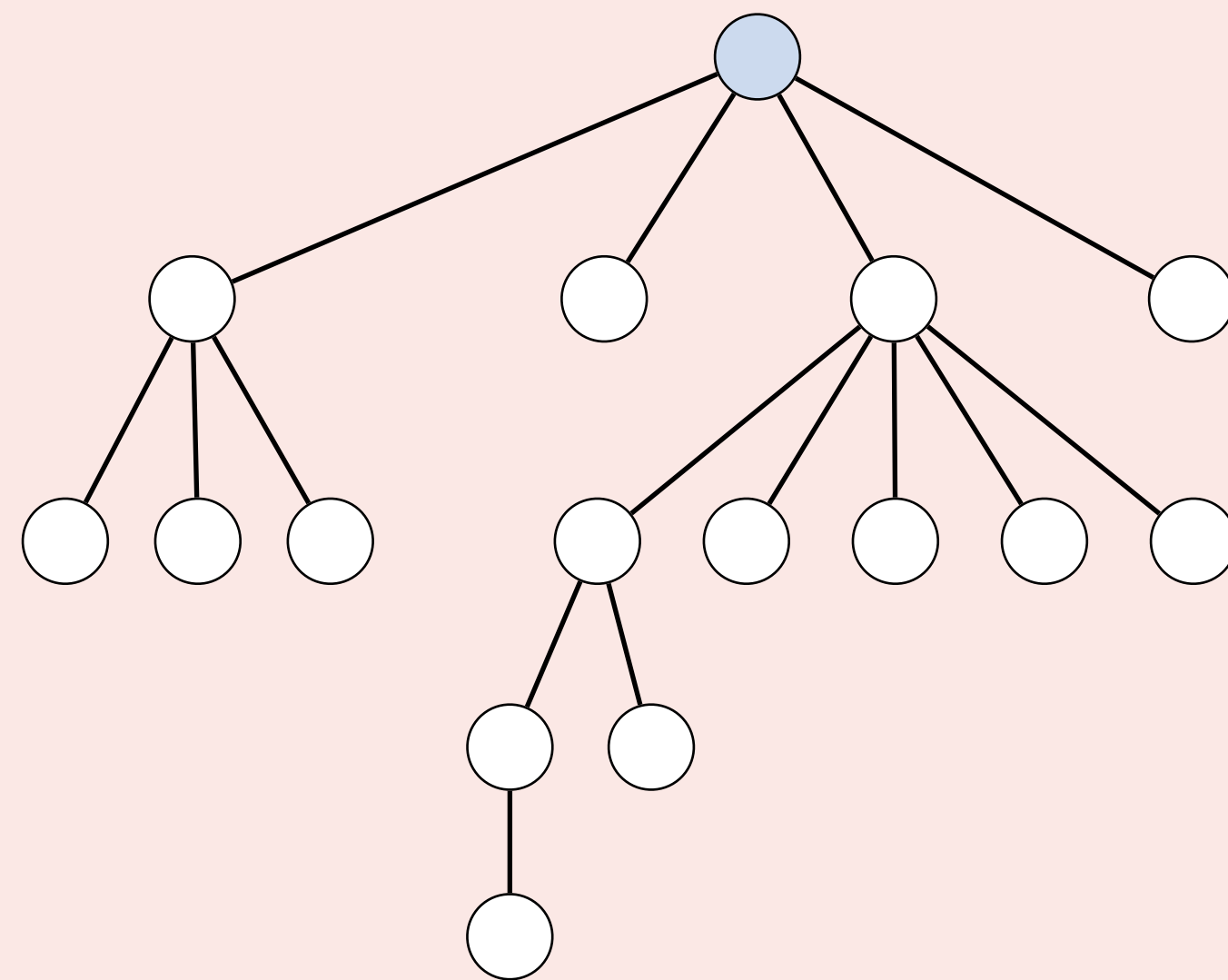
1.5 UNION-FIND

- *union-find data type*
- *quick-find*
- *quick-union*
- *weighted quick-union*

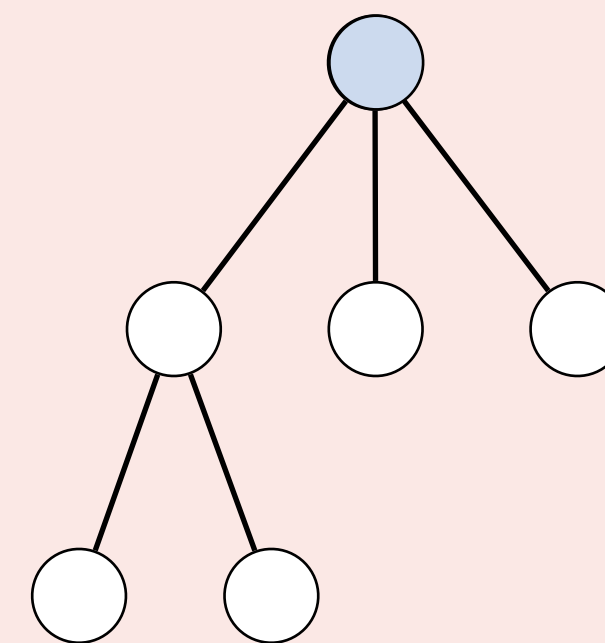


When linking two trees, which of these strategies is most effective?

- A. Link the root of the **smaller** tree to the root of the **larger** tree.
- B. Link the root of the **larger** tree to the root of the **smaller** tree.
- C. Flip a coin; randomly choose between A and B.
- D. All of the above.



larger tree
(size = 16, height = 4)



smaller tree
(size = 6, height = 2)

Weighted quick-union (link-by-size)

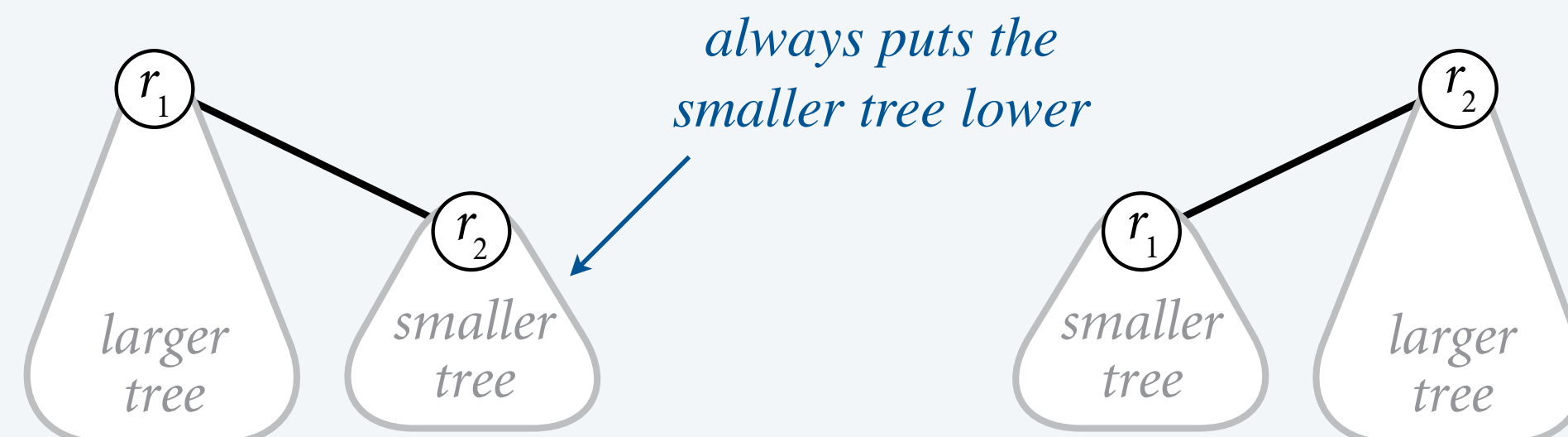
Link-by-size. Modify quick-union to avoid tall trees.

- Keep track of **size** of each tree = number of elements.
- Always link root of smaller tree to root of larger tree. ← *fine alternative: link-by-height (minimize worst-case depth vs. average depth)*

quick-union



weighted



Weighted quick-union: Java implementation

Data structure. Same as quick-union, but maintain extra array `size[i]`

to count number of elements in the tree rooted at `i`, initially 1.

- `find()`: identical to quick-union.
- `union()`: link root of smaller tree to root of larger tree; update `size[]`.

注意！当size相等的时候也要交换！
所以本来是p挂到q，当size相等的时候就会变成q挂到p！

```
public void union(int p, int q) {  
    int root1 = find(p);  
    int root2 = find(q);  
    if (root1 == root2) return;
```

```
    if (size[root1] >= size[root2]) {  
        int temp = root1; root1 = root2; root2 = temp;  
    }
```

```
    parent[root1] = root2;  
    size[root2] += size[root1];
```

```
}
```

link root of smaller tree
to root of larger tree

update size

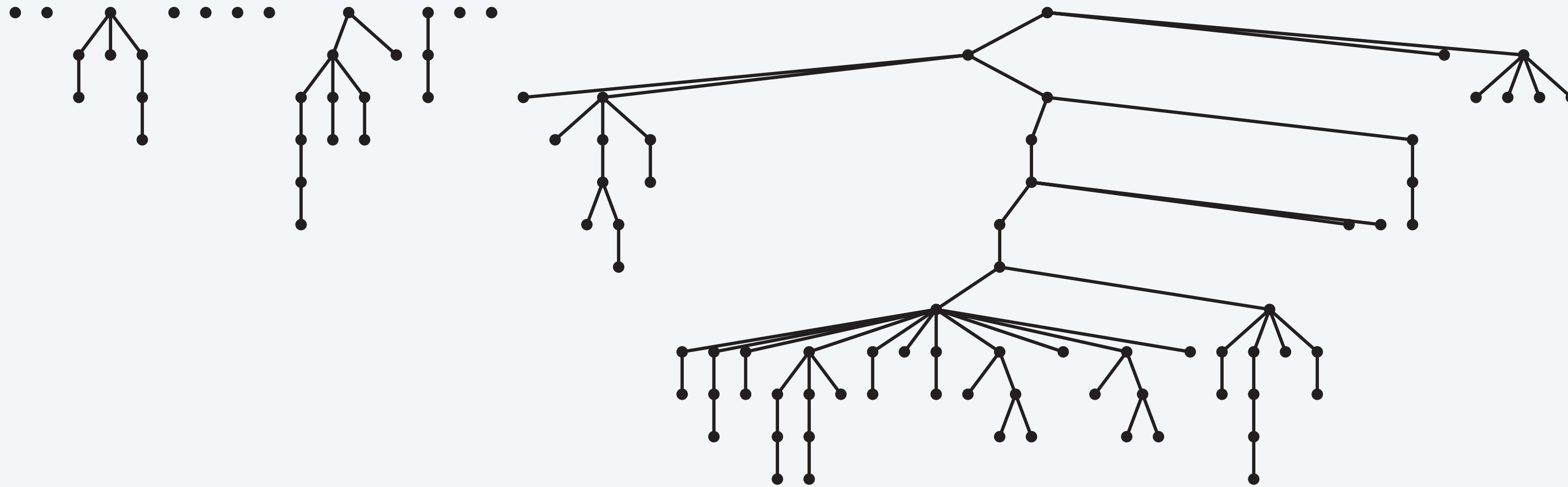
afterwards, root1
is root of smaller tree

这一步是交换，交换后，我们要保证：
root1 永远指向较小树的根，root2 永远指向较大树的根，
这样下面就可以固定将root1的父亲改成root2

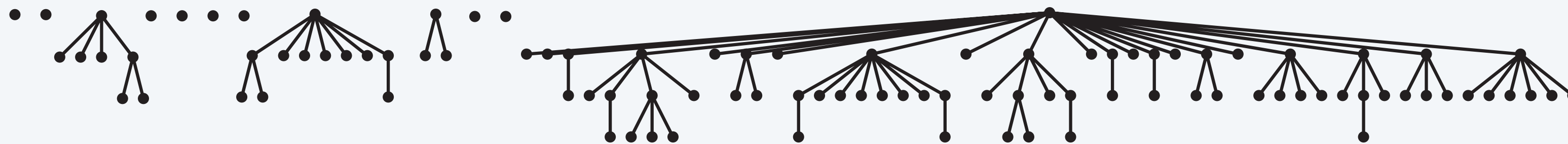
加权的union就是将size小的根的父亲改成size大的那个

Quick-union vs. weighted quick-union: larger example

quick-union

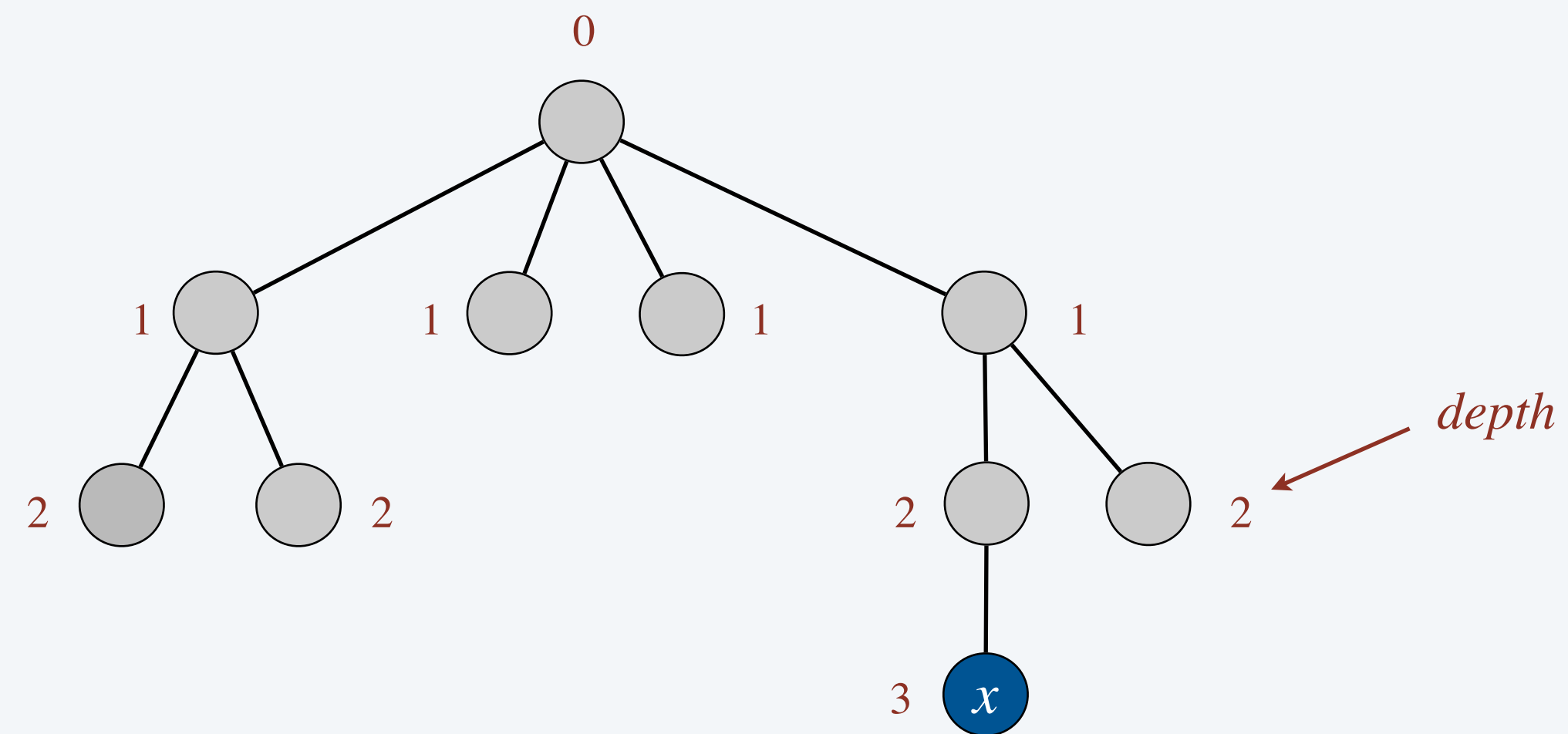


weighted



Weighted quick-union analysis

Proposition. Depth of any node $x \leq \log_2 n$.



$n = 10$
 $\text{depth}(x) = 3 \leq \log_2 n$

Weighted quick-union analysis

Proposition. Depth of any node $x \leq \log_2 n$.

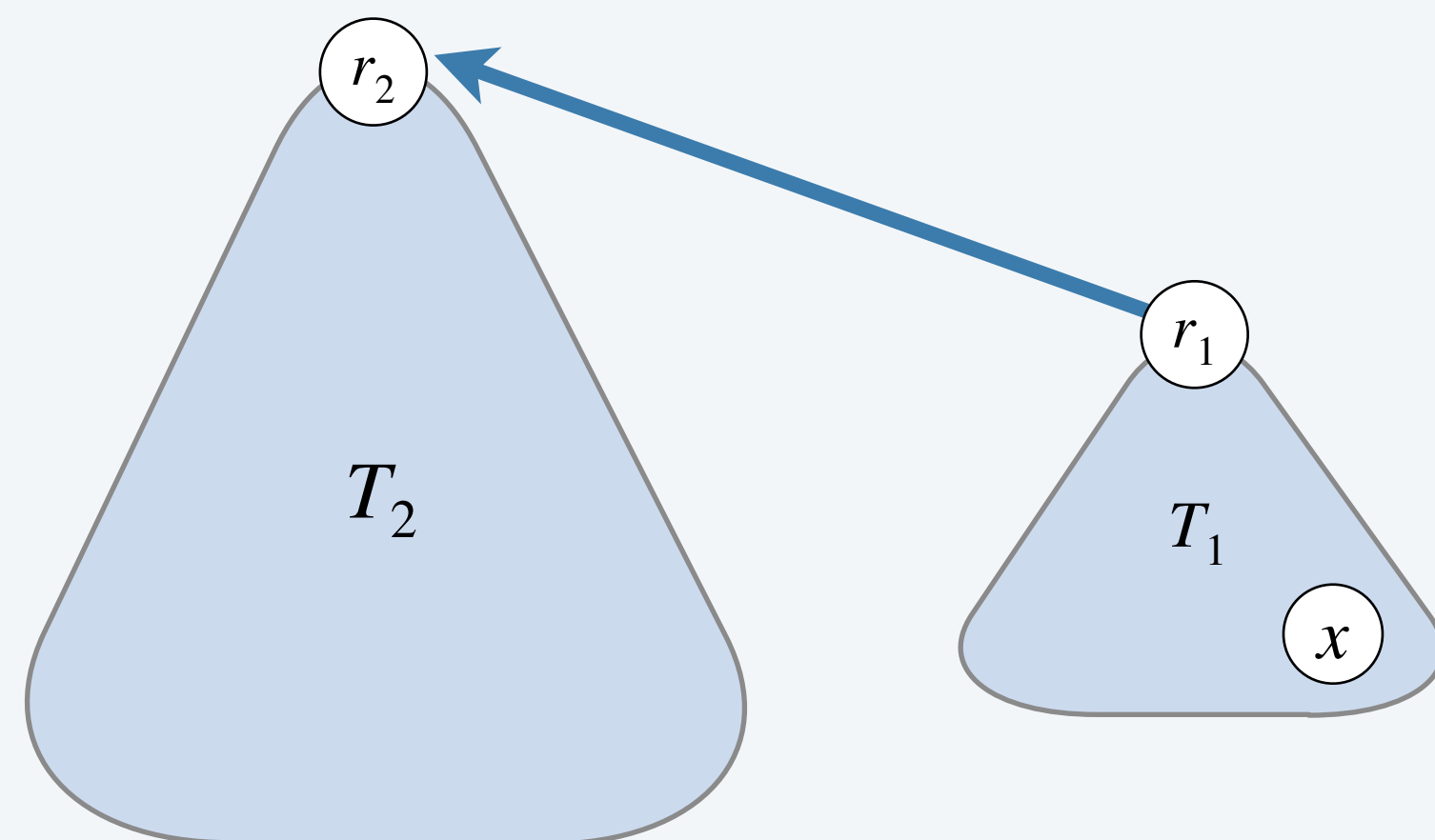
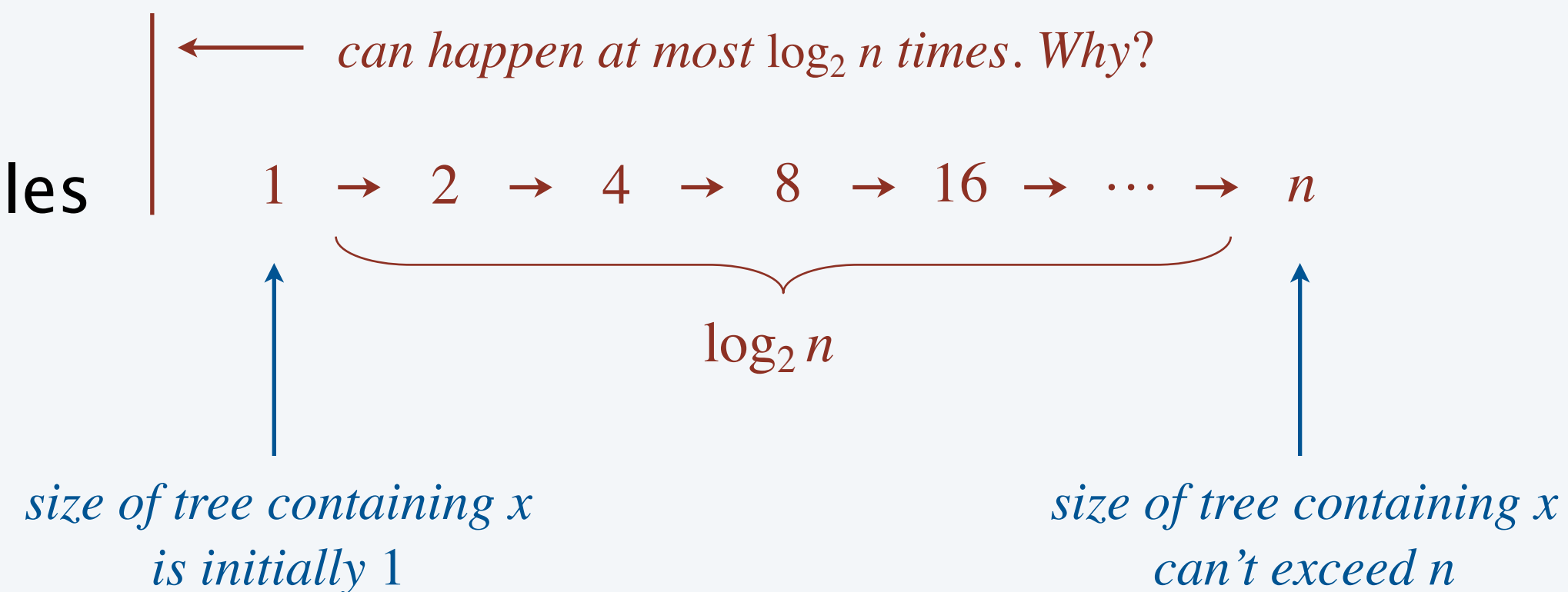
Pf.

- Depth of x does not change unless root of tree T_1 containing x is linked to the root of a larger tree T_2 , forming a new tree T_3 .

- When this happens:

- depth of x increases by exactly 1
- size of tree containing x at least doubles

because $\text{size}(T_3) = \text{size}(T_1) + \text{size}(T_2)$
 $\geq 2 \times \text{size}(T_1)$.



Weighted quick-union analysis

Proposition. Depth of any node $x \leq \log_2 n$.

Running time.

- `union()` takes constant time, given two roots.
- `find()` takes time proportional to **depth** of node in tree.

algorithm	initialize	union	find
quick-find	n	n	1
quick-union	n	n	n
weighted quick-union	n	$\log n$	$\log n$

← *in this course, log mean logarithm
for some constant base*

worst-case number of array accesses (ignoring leading coefficient)

Summary

Key point. Weighted quick-union empowers us to solve problems that could not otherwise be addressed.

algorithm	worst-case time
quick-find	$m n$
quick-union	$m n$
weighted quick-union	$m \log n$
quick-union + path compression	$m \log n$ ← fastest for percolation?
weighted quick-union + path compression	$m \alpha(m, n)$ ← inverse Ackermann function (see Turing precept or COS 423)

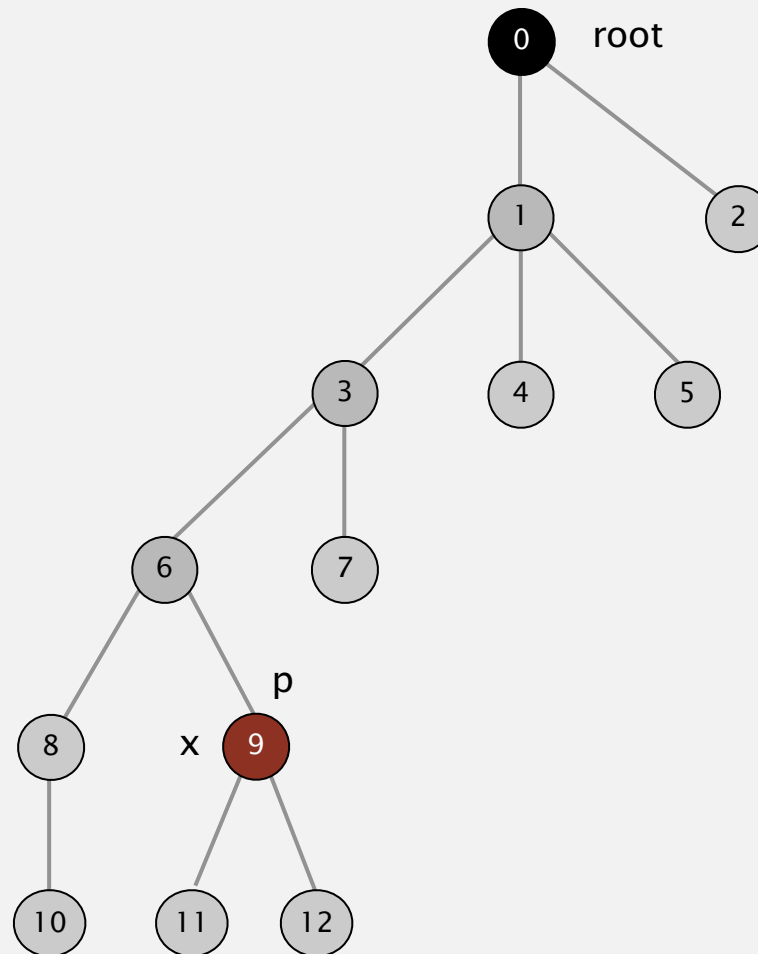
order of growth for $m \geq n$ union-find operations on a set of n elements

Ex. [10^9 union-find operations on 10^9 elements]

- Efficient algorithm reduces time from 30 years to 6 seconds.
- Supercomputer won't help much.

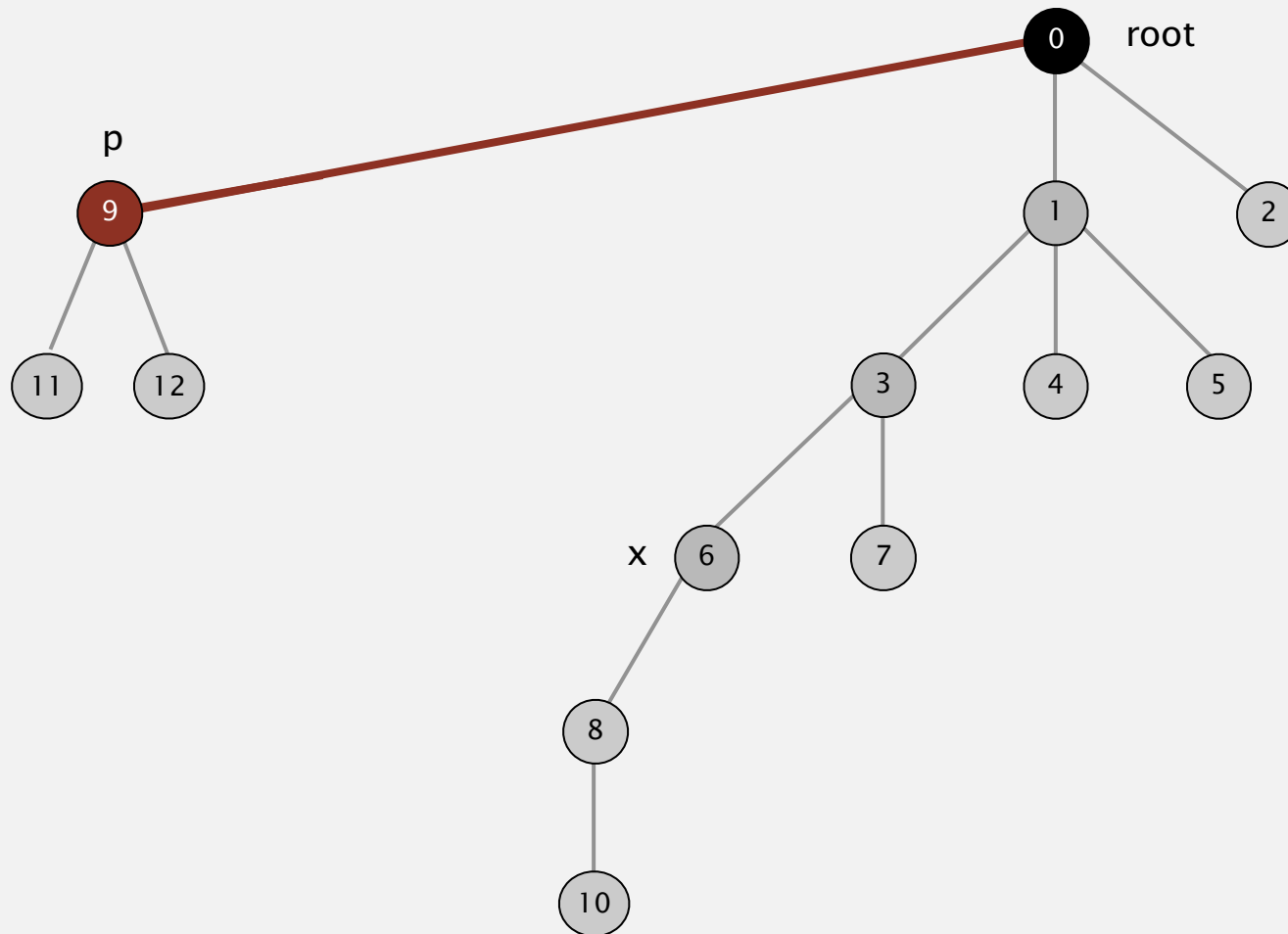
Improvement 2: path compression

Quick union with path compression. Just after computing the root of p , set the id of each examined node to point to that root.



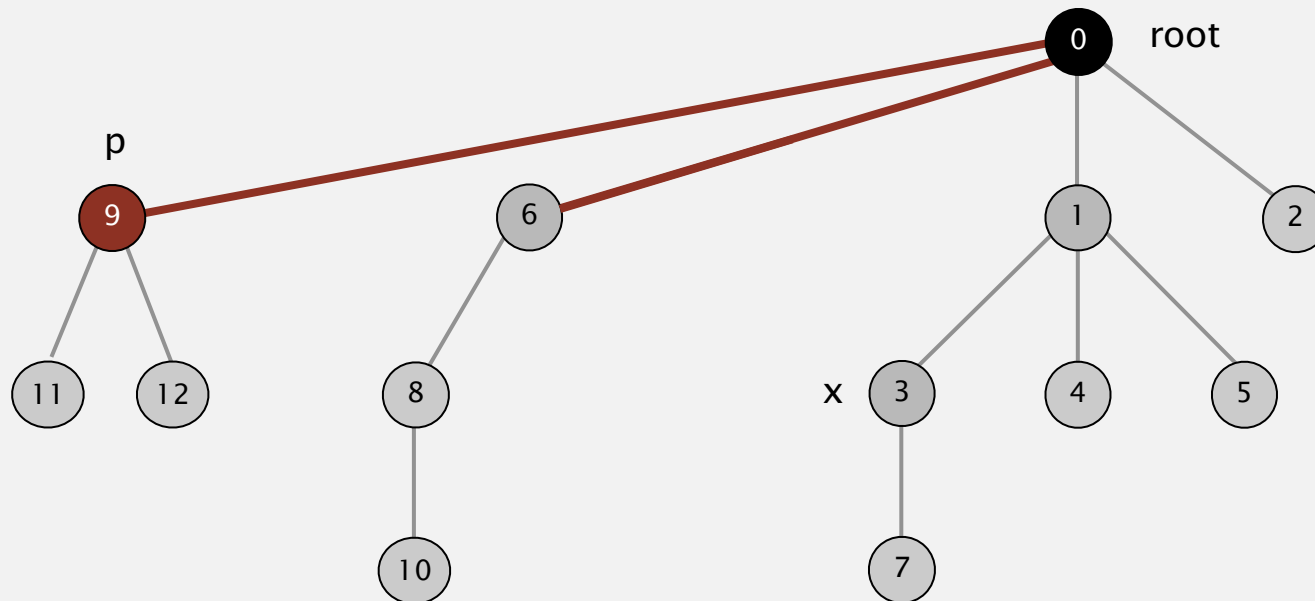
Improvement 2: path compression

Quick union with path compression. Just after computing the root of p , set the id of each examined node to point to that root.



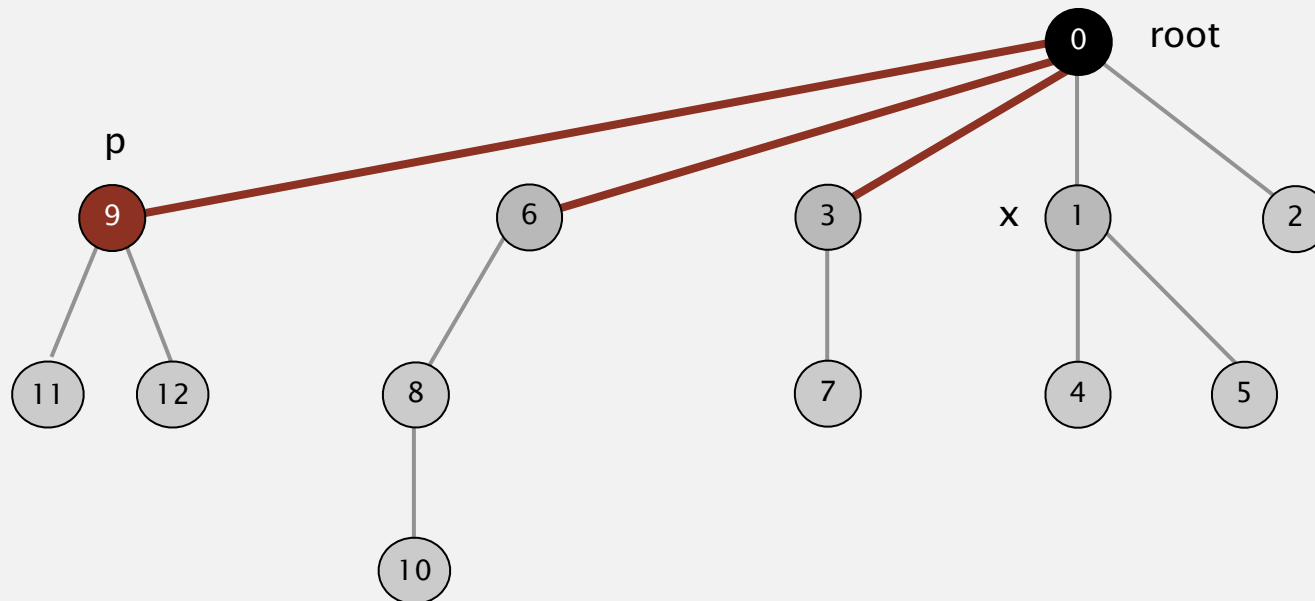
Improvement 2: path compression

Quick union with path compression. Just after computing the root of p , set the id of each examined node to point to that root.



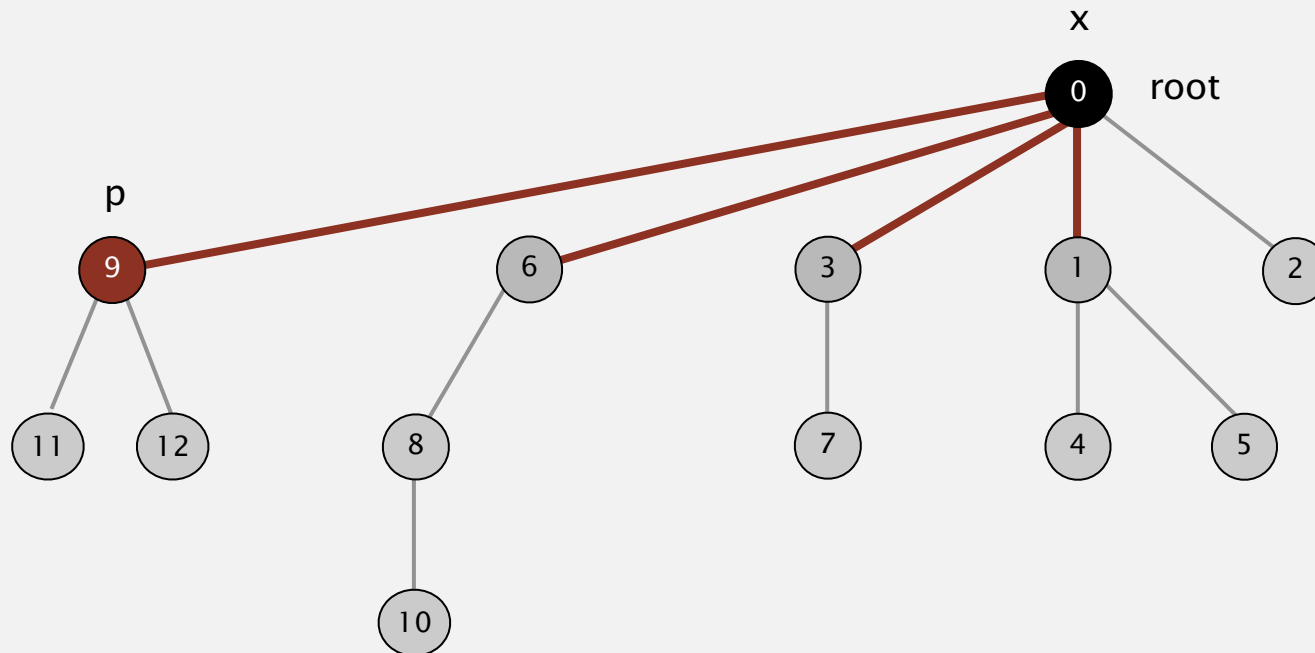
Improvement 2: path compression

Quick union with path compression. Just after computing the root of p , set the id of each examined node to point to that root.



Improvement 2: path compression

Quick union with path compression. Just after computing the root of p , set the id of each examined node to point to that root.



Path compression: Java implementation

Two-pass implementation: add **second loop** to root() to set the id[] of each examined node to the root.

Simpler one-pass variant: Make every other node in path point to its grandparent (thereby halving path length).

```
private int root(int i)
{
    while (i != id[i])
    {
        id[i] = id[id[i]];
        i = id[i];
    }
    return i;
}
```

← only one extra line of code !

In practice. No reason not to! Keeps tree almost completely flat.

Weighted quick-union with path compression: amortized analysis

Proposition. [Hopcroft-Ulman, Tarjan] Starting from an empty data structure, any sequence of M union-find ops on N objects makes $\leq c (N + M \lg^* N)$ array accesses.

- Analysis can be improved to $N + M \alpha(M, N)$.
- Simple algorithm with fascinating mathematics.

N	$\lg^* N$
1	0
2	1
4	2
16	3
65536	4
2^{65536}	5

iterate log function

Linear-time algorithm for M union-find ops on N objects?

- Cost within constant factor of reading in the data.
- In theory, WQUPC is not quite linear.
- In practice, WQUPC is linear.

Amazing fact. [Fredman-Saks] No linear-time algorithm exists.


in "cell-probe" model of computation

Summary

Bottom line. Weighted quick union (with path compression) makes it possible to solve problems that could not otherwise be addressed.

algorithm	worst-case time
quick-find	$M N$
quick-union	$M N$
weighted QU	$N + M \log N$
QU + path compression	$N + M \log N$
weighted QU + path compression	$N + M \lg^* N$

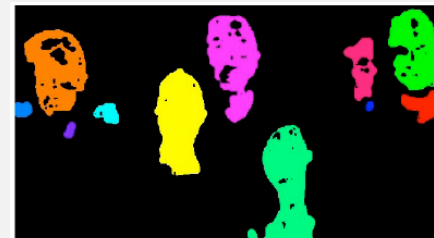
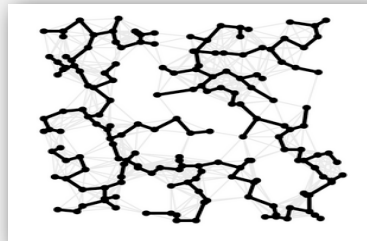
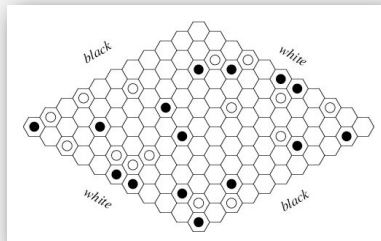
M union-find operations on a set of N objects

Ex. [10^9 unions and finds with 10^9 objects]

- WQUPC reduces time from 30 years to 6 seconds.
- Supercomputer won't help much; good algorithm enables solution.

Union-find applications

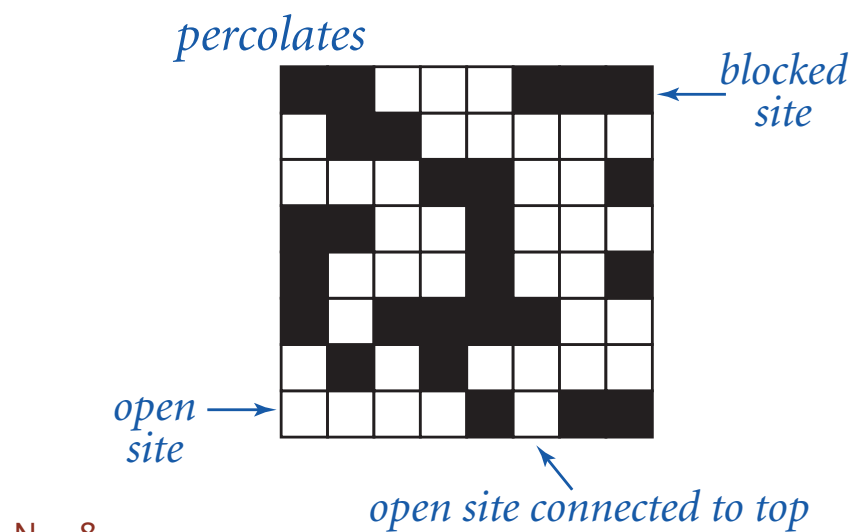
- Percolation.
- Games (Go, Hex).
- ✓ Dynamic connectivity.
 - Least common ancestor.
 - Equivalence of finite state automata.
 - Hoshen-Kopelman algorithm in physics.
 - Hinley-Milner polymorphic type inference.
 - Kruskal's minimum spanning tree algorithm.
 - Compiling equivalence statements in Fortran.
 - Morphological attribute openings and closings.
 - Matlab's `bwlabel()` function in image processing.



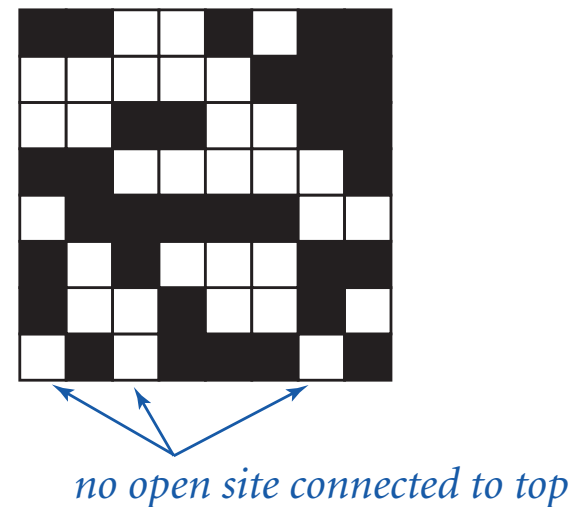
Percolation

A model for many physical systems:

- N -by- N grid of sites.
- Each site is open with probability p (or blocked with probability $1 - p$).
- System **percolates** iff top and bottom are connected by open sites.



does not percolate



Percolation

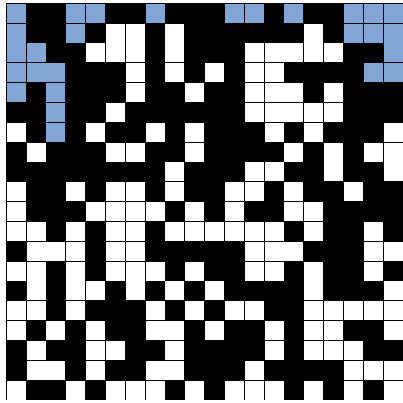
A model for many physical systems:

- N -by- N grid of sites.
- Each site is open with probability p (or blocked with probability $1 - p$).
- System **percolates** iff top and bottom are connected by open sites.

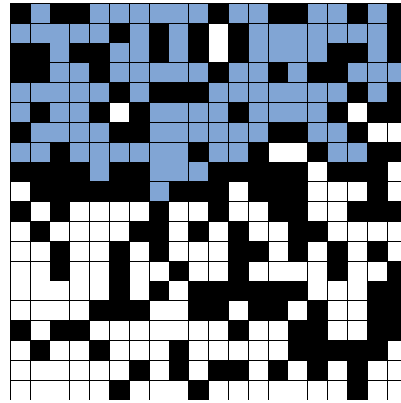
model	system	vacant site	occupied site	percolates
electricity	material	conductor	insulated	conducts
fluid flow	material	empty	blocked	porous
social interaction	population	person	empty	communicates

Likelihood of percolation

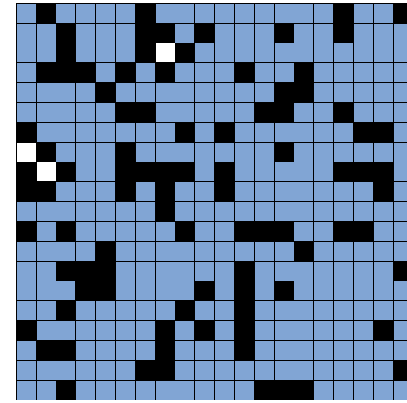
Depends on site vacancy probability p .



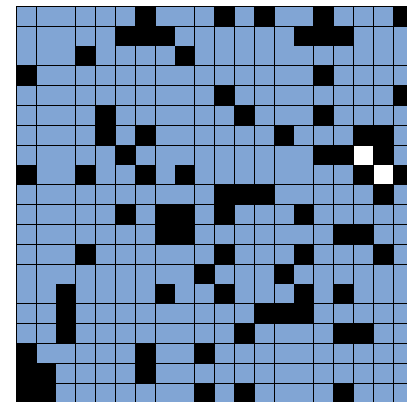
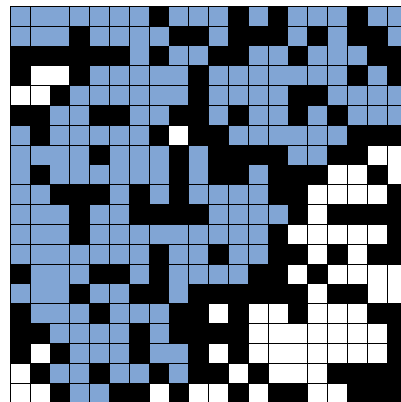
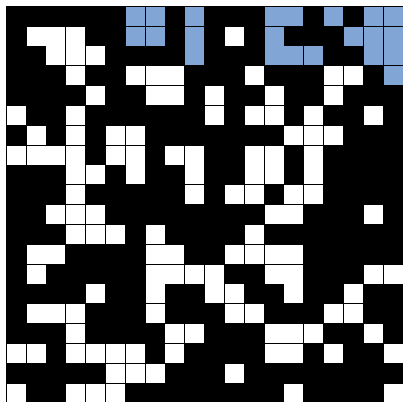
p low (0.4)
does not percolate



p medium (0.6)
percolates?



p high (0.8)
percolates

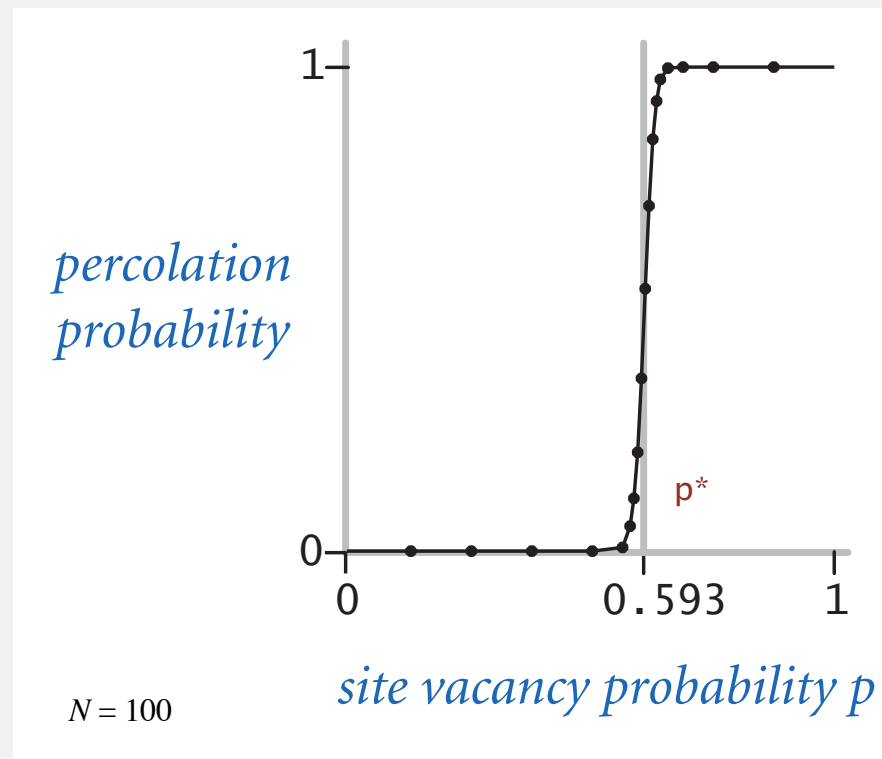


Percolation phase transition

When N is large, theory guarantees a sharp threshold p^* .

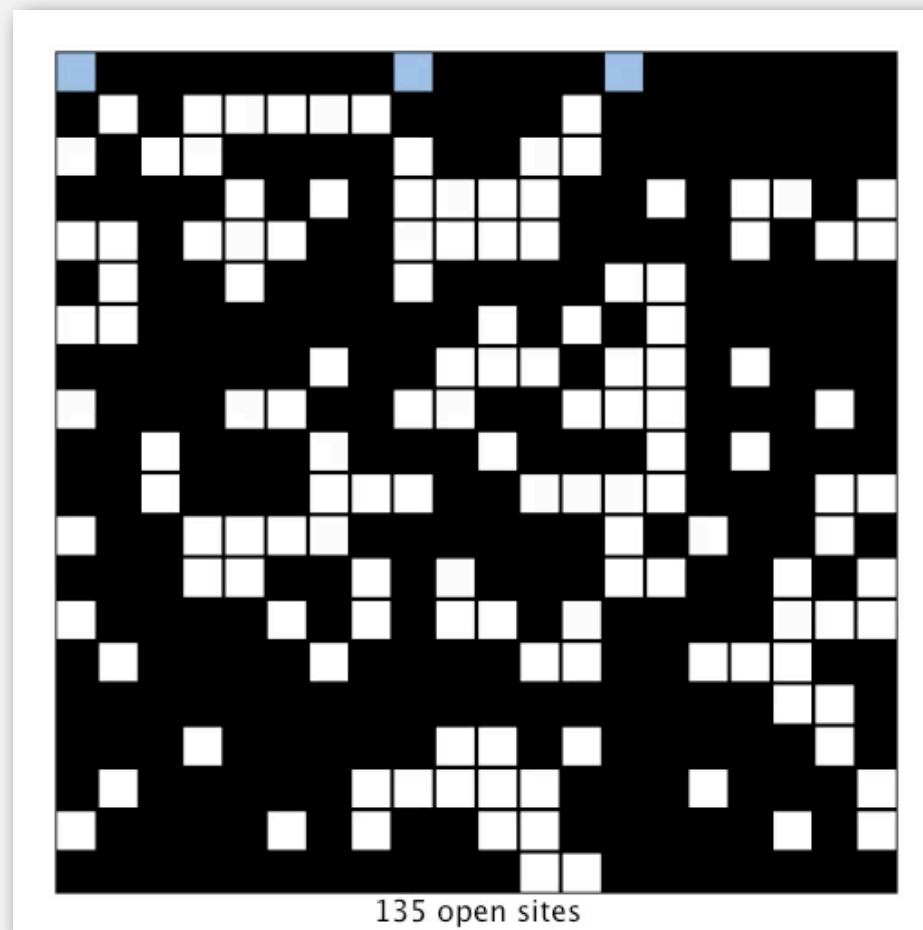
- $p > p^*$: almost certainly percolates.
- $p < p^*$: almost certainly does not percolate.

Q. What is the value of p^* ?

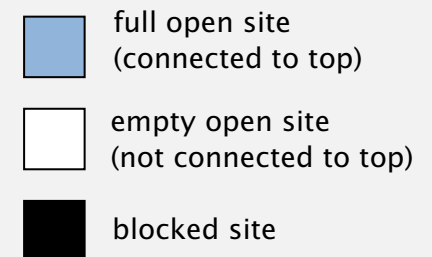


Monte Carlo simulation

- Initialize N -by- N whole grid to be blocked.
- Declare random sites open until top connected to bottom.
- Vacancy percentage estimates p^* .



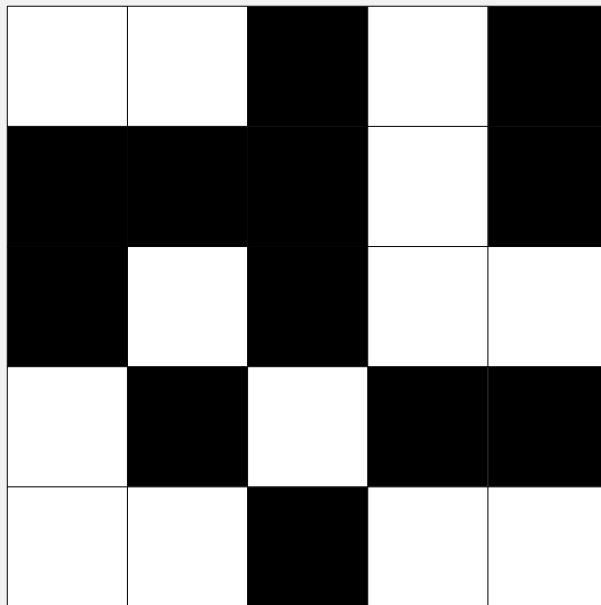
$N = 20$



Dynamic connectivity solution to estimate percolation threshold

Q. How to check whether an N -by- N system percolates?

$N = 5$



open site

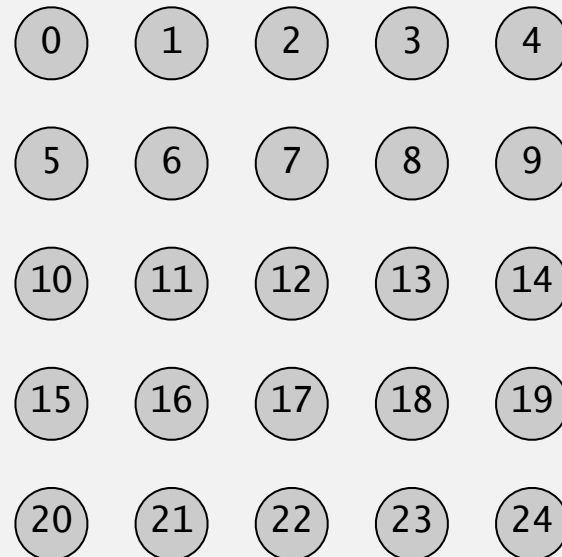
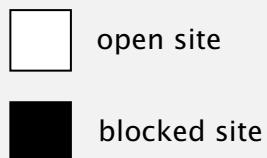
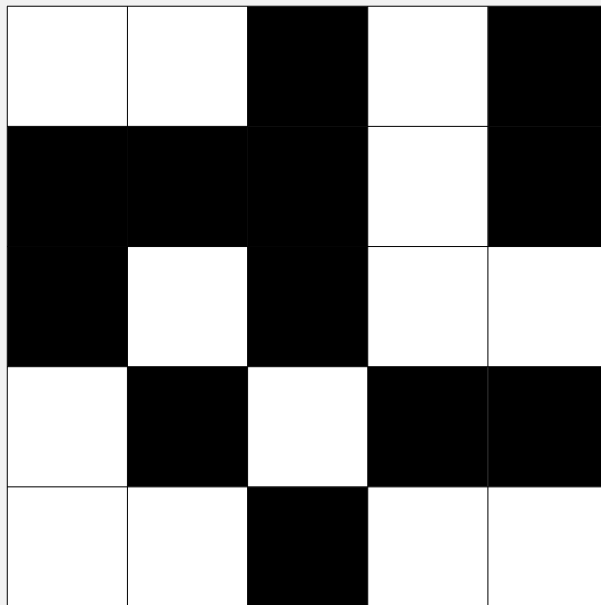


blocked site

Dynamic connectivity solution to estimate percolation threshold

- Q. How to check whether an N -by- N system percolates?
- Create an object for each site and name them 0 to $N^2 - 1$.

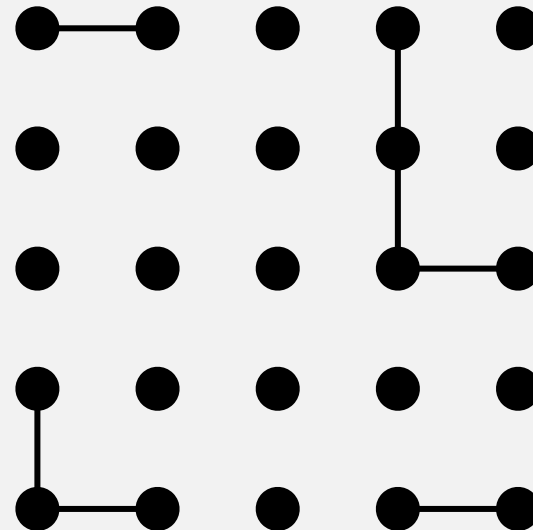
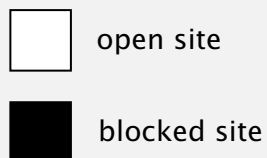
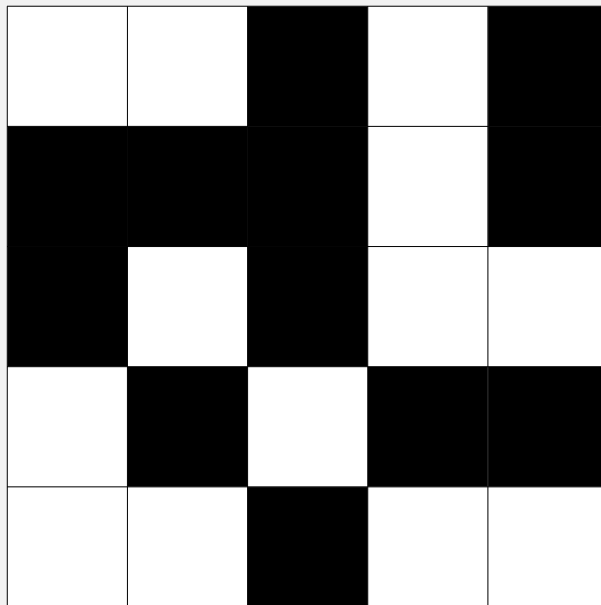
$N = 5$



Dynamic connectivity solution to estimate percolation threshold

- Q. How to check whether an N -by- N system percolates?
- Create an object for each site and name them 0 to $N^2 - 1$.
 - Sites are in same component if connected by open sites.

$N = 5$



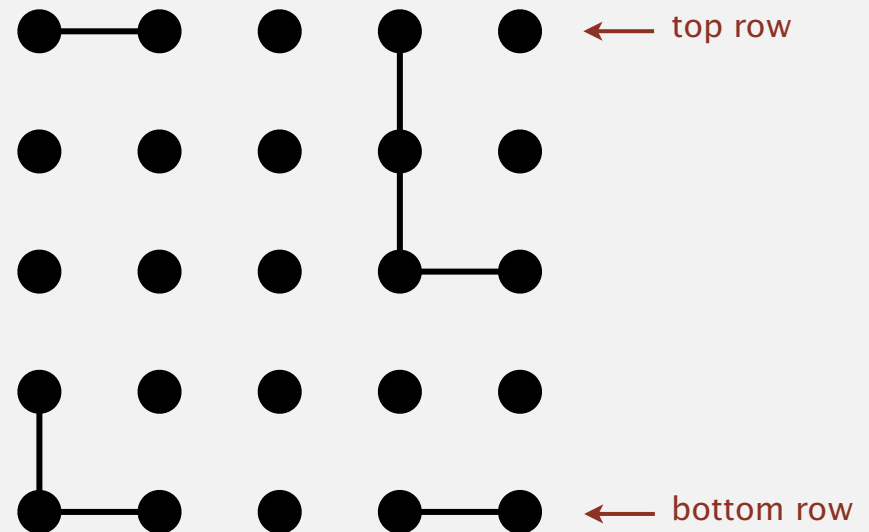
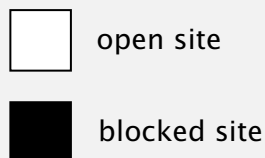
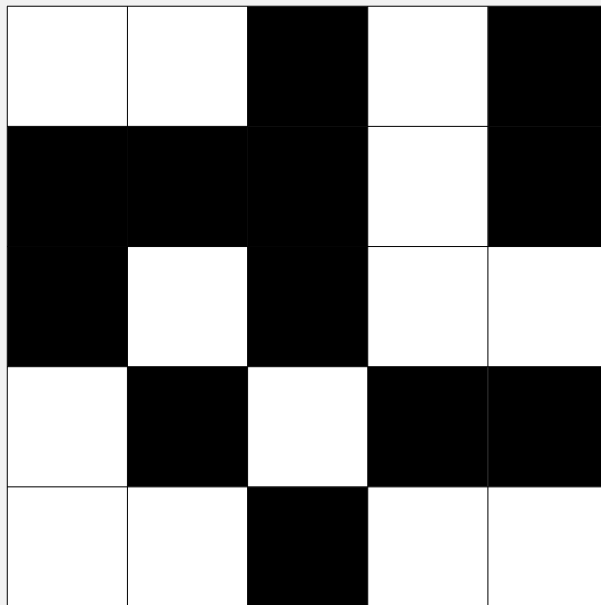
Dynamic connectivity solution to estimate percolation threshold

Q. How to check whether an N -by- N system percolates?

- Create an object for each site and name them 0 to $N^2 - 1$.
- Sites are in same component if connected by open sites.
- Percolates iff any site on bottom row is connected to site on top row.

brute-force algorithm: N^2 calls to `connected()`

$N = 5$



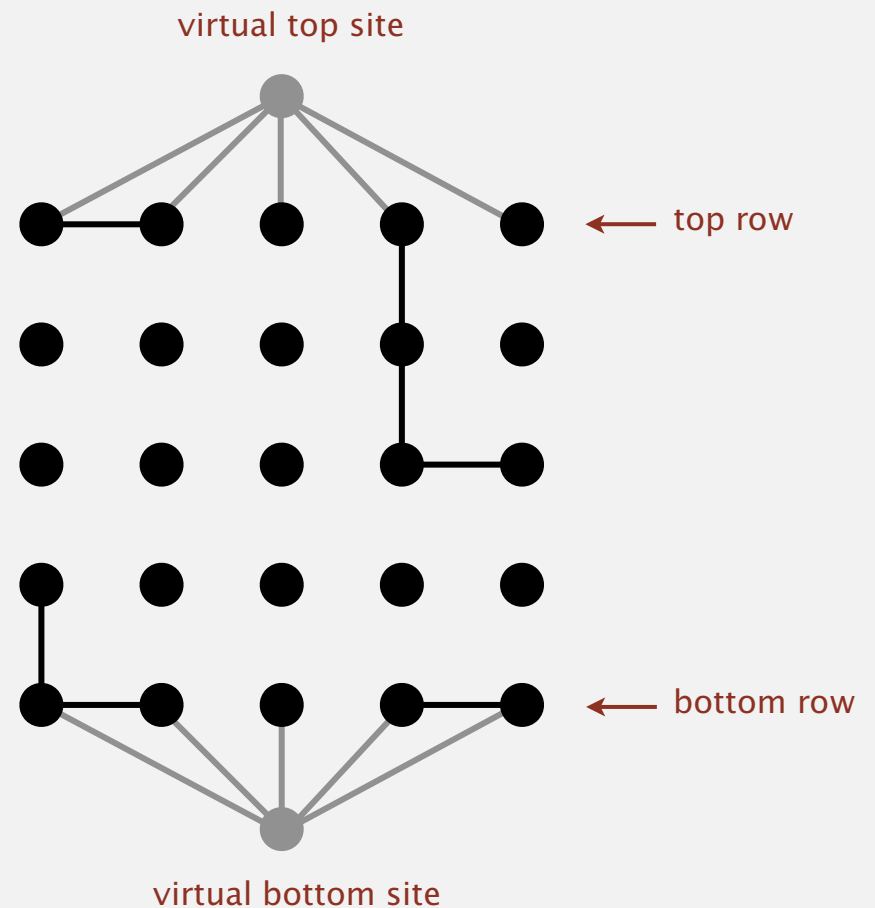
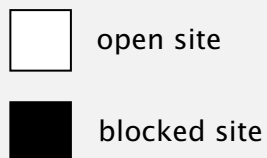
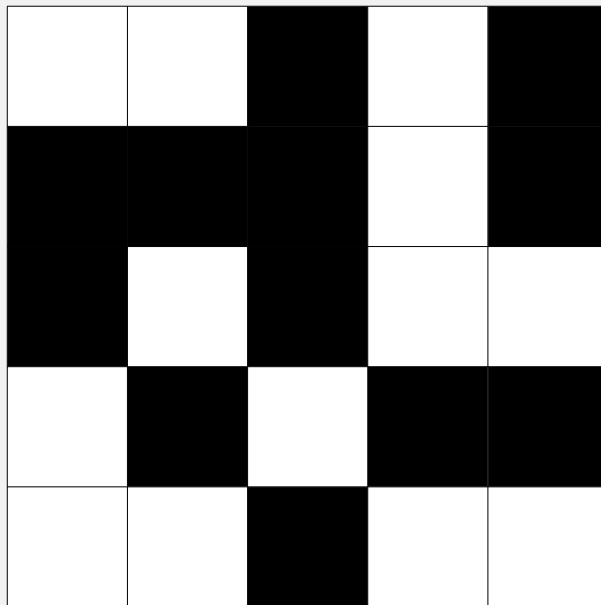
Dynamic connectivity solution to estimate percolation threshold

Clever trick. Introduce 2 virtual sites (and connections to top and bottom).

- Percolates iff virtual top site is connected to virtual bottom site.

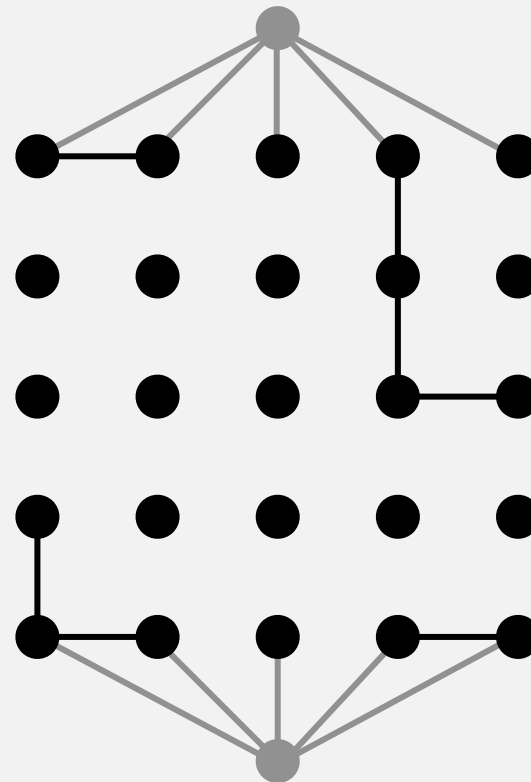
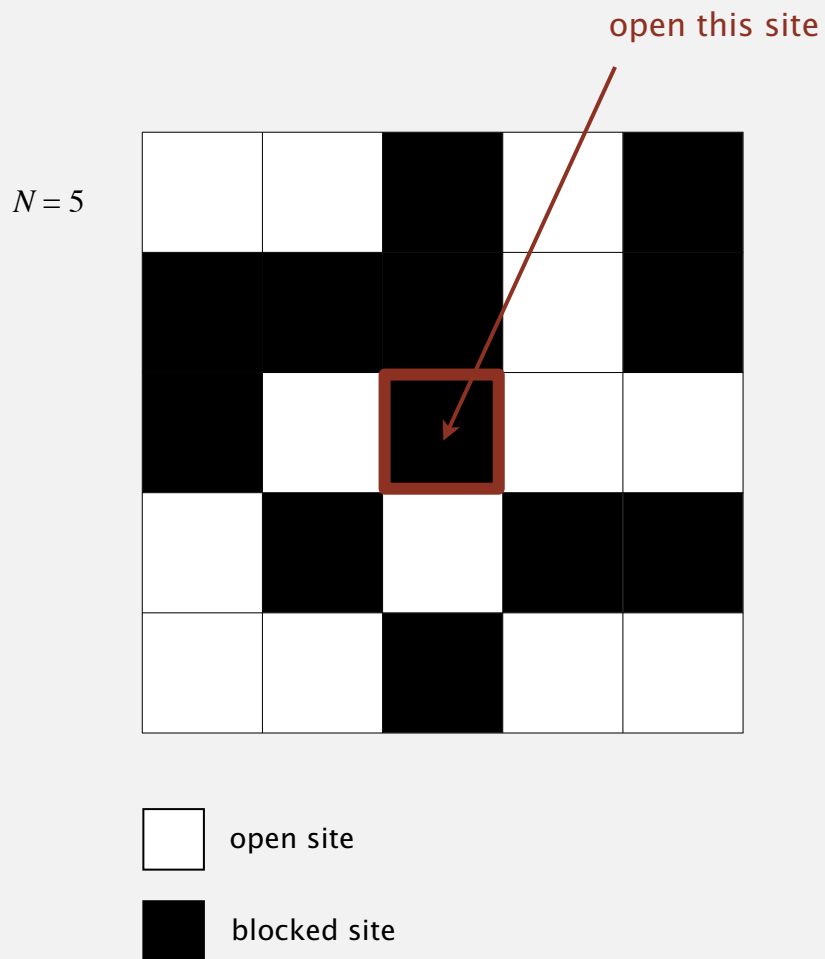
efficient algorithm: only 1 call to connected()

$N = 5$



Dynamic connectivity solution to estimate percolation threshold

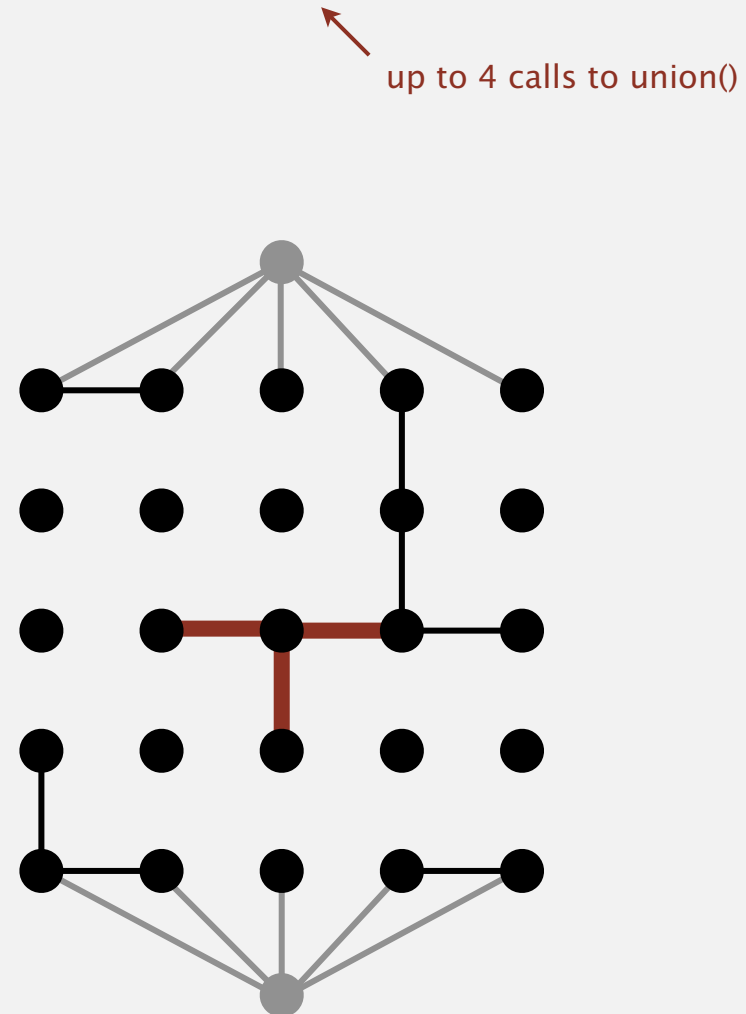
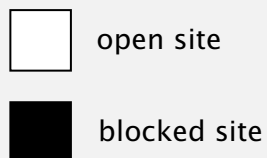
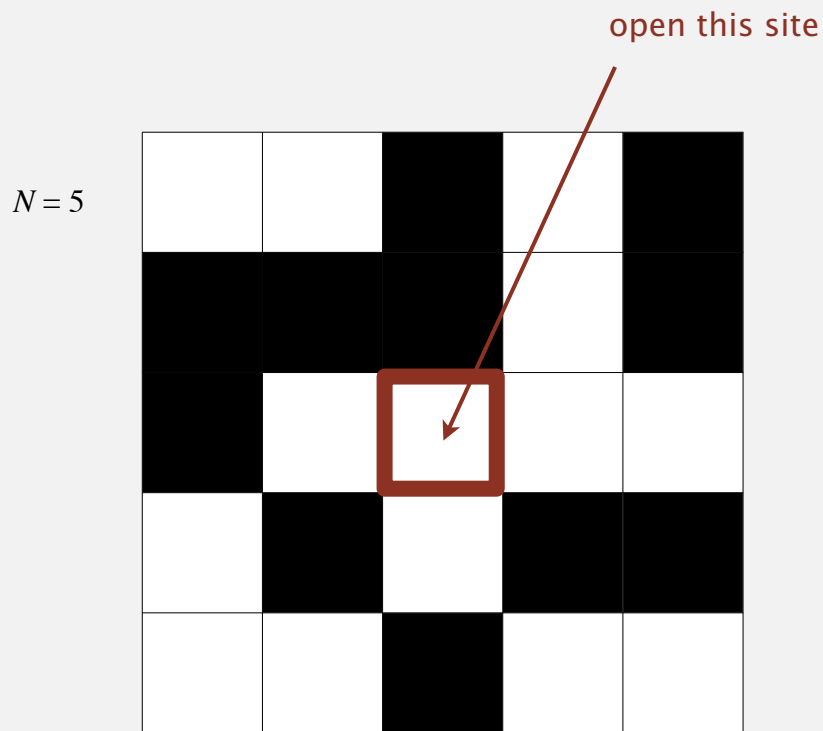
Q. How to model opening a new site?



Dynamic connectivity solution to estimate percolation threshold

Q. How to model opening a new site?

A. Mark new site as open; connect it to all of its adjacent open sites.

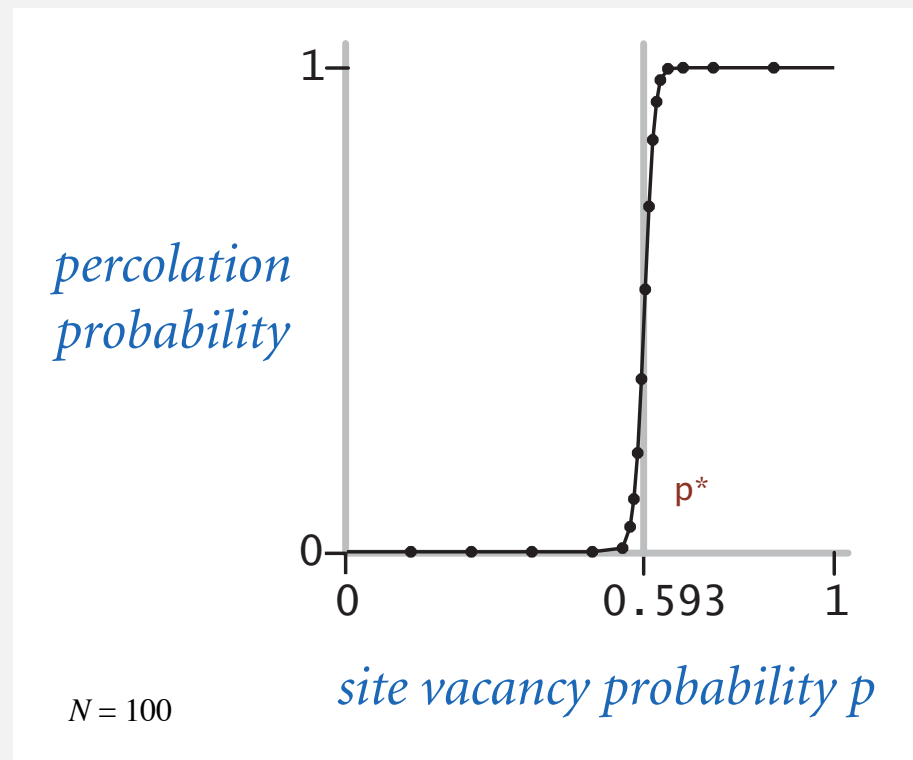


Percolation threshold

Q. What is percolation threshold p^* ?

A. About 0.592746 for large square lattices.

constant known only via simulation



Fast algorithm **enables** accurate answer to scientific question.



1.5 UNION-FIND

- ▶ *dynamic connectivity*
- ▶ *quick find*
- ▶ *quick union*
- ▶ *improvements*
- ▶ *applications*

Subtext of today's lecture (and this course)

Steps to developing a usable algorithm.

- Model the problem.
- Find an algorithm to solve it.
- Fast enough? Fits in memory?
- If not, figure out why.
- Find a way to address the problem.
- Iterate until satisfied.

The scientific method.

Mathematical analysis.

Credits

image	source
<i>Game of Hex</i>	<u>Wolfram MathWorld</u>
<i>Cluster Labeling</i>	<u>Tiberiu Marita</u>
<i>Bob Tarjan</i>	<u>Princeton University</u>
<i>Computer and Supercomputer</i>	<u>New York Times</u>

A final thought



*“The goal is to come up with algorithms that you can apply in practice that **run fast**, as well as being **simple, beautiful, and analyzable**.”* — Robert Tarjan

